Laporan Milestone 1: Sistem Terdistribusi Dasar

Tugas Besar IF4031 Arsitektur Aplikasi Terdistribusi

Sistem Koordinasi Bahaya



Disusun oleh:

AAAAAAAAAAT

13523090	Nayaka Ghana Subrata
13524003	Faiq Azzam Nafidz
13524013	Anindya Naufal Pinasthika

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2026

Daftar Isi

Daftar Isi **1**
Deskripsi Umum **2**
Arsitektur Sistem **3**
Diagram sistem 3
Penjelasan Tiap Komponen beserta Protokol Komunikasi & Kepemilikan 3
Alur Event 8
Batas Kontainer 13
Teknologi yang Digunakan **14**
1. Bahasa Pemrograman 14
2. Database 14
3. Message Broker 14
Asumsi **16**
Petunjuk Menjalankan Sistem **17**
Penyelesaian Problem **18**
Problem 1: Arsitektur & Interoperabilitas Data 18
Problem 2: Concurrency, Availability, & Graceful Degradation 22
Problem 3: Service-to-Service Authentication & Trust 29
Problem 4: Arsitektur Microservice & Flexible Storage 47
Problem 5: Fan-out Event Distribution/Pub-sub 55
Referensi **72**
Lampiran **73**

Deskripsi Umum

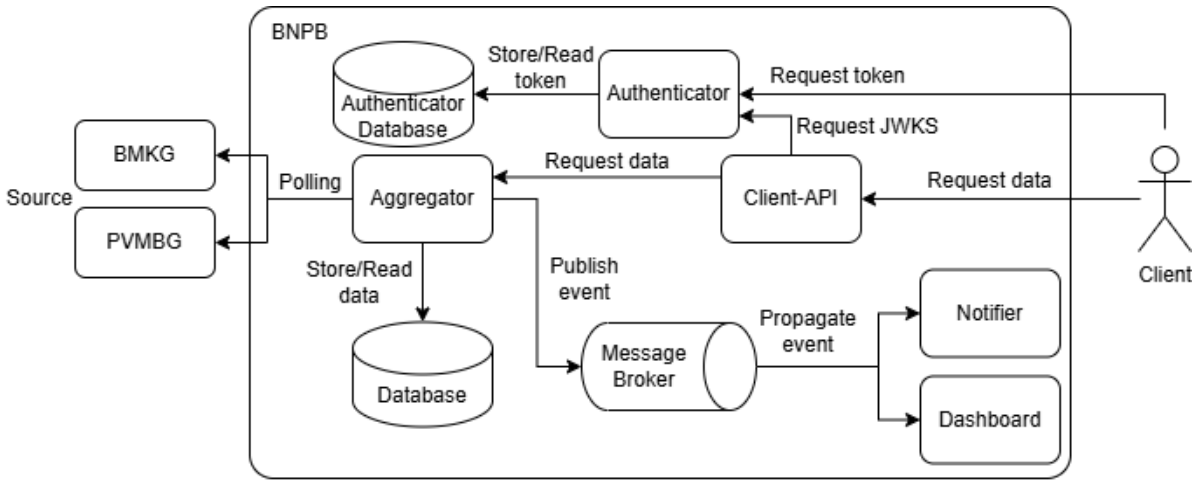
Pada tugas besar ini meminta untuk merancang dan mengimplementasikan arsitektur aplikasi terdistribusi (kumpulan komponen independen yang saling bekerja sama melalui jaringan untuk mencapai tujuan bersama) untuk domain penanggulangan bencana. Milestone 1 berfokus pada pembangunan fondasi aplikasi agar beberapa lembaga-lembaga otonom yaitu BMKG (memantau gempa, cuaca, dan potensi tsunami), PVMBG (memantau aktivitas gunung api), dan BNPB (mengoordinasi respons dan pengambilan keputusan lintas wilayah) bisa saling berkomunikasi lewat antarmuka yang jelas, menangani banyak permintaan tanpa saling memblokir, dan tetap berjalan wajar walau salah satu di antaranya melambat. Pengerjaan milestone 1 dilakukan dengan menjawab problem set yang diberikan untuk menentukan kebutuhan sistem.

Sistem yang dibuat pada tugas besar ini terdiri atas layanan dari BMKG, PVMBG, dan BPNB.

- BMKG sebagai instansi sumber, menyediakan layanan berupa *mock service* yang menyimulasikan pemberian data hasil pemantauan gempa dan potensi tsunami.
- PVMBG sebagai instansi sumber, menyediakan layanan berupa *mock service* yang menyimulasikan pemberian data hasil pemantauan gunung api.
- BPNB sebagai *node* koordinasi yang terdiri atas komponen-komponen berikut.
    - Gateway/Aggregator: melakukan *polling* ke BMKG dan PVMBG secara berkala untuk mendapat data bencana, memetakan data tersebut ke struktur data khusus lalu menyimpannya ke dalam basis data, dan mempublikasikan event berdasarkan data baru yang didapat dari BMKG dan PVMBG.
    - Database: menyimpan data bencana yang telah dipetakan oleh komponen Aggregator.
    - Client-Facing API: memberi *client* akses ke data bencana pada basis data dengan kelengkapan data sesuai hak aksesnya (lihat bagian authenticator).
    - Authenticator: menerbitkan token untuk *client* dengan *scope* yang berbeda sesuai hak akses (Media/Pers: ringkasan, Tim Lapangan: data mentah, BNPB Pusat/Internal Operators: data mentah).
    - Authenticator Database: menyimpan token yang diterbitkan authenticator.
    - Message Broker: menyalurkan data bencana dari *producer* (aggregator) ke *consumer* (event consumer).
    - Event Consumer berupa Dashboard Consumer dan Event Consumer.

Arsitektur Sistem

Diagram sistem



Gambar 1. Diagram Arsitektur Sistem

Desain arsitektur tersebut disesuaikan dengan komponen yang disebutkan pada spesifikasi: BMKG dan PVMBG sebagai instansi sumber terpisah dengan BNPB; BNPB terdiri dari aggregator, database, client-facing API, authenticator (beserta basis datanya), message broker, dan event consumer (notifier dan dashboard); dan client mengakses layanan BNPB melalui komponen client-facing API. Panah pada diagram tersebut merepresentasikan interaksi antarkomponen, dengan komponen yang menunjuk komponen lain berarti komponen tersebut yang mengirim pesan/request kepada komponen tersebut dan secara opsional komponen yang ditunjuk dapat memberi respon (berupa konfirmasi atau data yang diminta penunjuk).

Penjelasan Tiap Komponen beserta Protokol Komunikasi & Kepemilikan

Secara umum, komunikasi antarkomponen bersifat *request-response* dengan menggunakan protokol HTTP/REST. Protokol ini digunakan karena kecukupannya dalam menyediakan hal-hal yang dibutuhkan sistem dengan kesederhanaannya dalam pembuatan dan pemahaman kode implementasi. Berikut adalah penjelasan komponen yang terdapat pada sistem beserta protokol komunikasi yang diterapkan serta kepemilikannya jika merupakan komponen penyimpanan.

1. **BMKG**

   Komponen BMKG menjadi instansi sumber yang memberikan data berupa SeismicEvent dan TsunamiWarning. Komponen berjalan pada port 8081 dengan *endpoint*:

   - GET /seismic-events?since=<timestamp>
   - GET /tsunami-warnings?since=<timestamp>
   - GET /health

   Request memerlukan *header field* X-BMKG-Key sebagai autentikasi menggunakan API key serta *field* X-Correlation-ID berisi waktu dimunculkannya *request* sebagai pelacak permintaan.

2. **PVMBG**

   Komponen PVMBG juga menjadi instansi sumber yang memberikan data VolcanicReport. Komponen berjalan pada port 8082 dengan *endpoint*:

   - GET /volcanic-reports?since=<timestamp>
   - GET /health
   - POST /admin/schema-version
   - POST /admin/outage

   Request memerlukan *header field* Authorization sebagai autentikasi menggunakan token PVMBG yang berbeda dengan autentikasi BMKG serta *field* X-Correlation-ID berisi waktu dimunculkannya *request* sebagai pelacak permintaan.

3. **Aggregator**

   Aggregator melakukan *polling* data dari instansi sumber, mengagregasi data dengan memetakannya menjadi HazardEvent lalu menyimpannya dalam komponen database, mengambil data dari database, dan mempublikasikan *event* yang berisi data HazardEvent baru yang diperoleh ke komponen message broker. Komponen berjalan pada port 8083 dengan *endpoint*:

   - GET /health
   - GET /internal/events

- GET /internal/events/{id}
- GET /internal/source-status

Request memerlukan *header field* X-BMKG-Key sebagai autentikasi menggunakan API key.

4. Database

Komponen ini dimiliki sepenuhnya oleh aggregator untuk menyimpan data HazardEvent yang diperoleh oleh aggregator. Database diimplementasikan dengan PostgreSQL yang berjalan pada port 5432 serta hanya dapat diakses melalui *network* khusus bernama database (berisi komponen database dan aggregator saja). Isi dari database adalah satu tabel bernama hazard_events yang atributnya sama seperti atribut pada HazardEvent yang dihasilkan aggregator.

5. Authenticator

Komponen authenticator membangkitkan token atau membaca token yang sudah ada lalu diberikan ke *client downstream* yang bersesuaian. Terdapat tingkatan hak akses *client* yang didasarkan pada tiga identitas *client*: media-client (mendapat *field* ringkasan), field-client (mendapat seluruh *field*, token ber-TTL pendek), ops-client (mendapat seluruh *field*, hak akses tertinggi). Implementasi autentikasi dibuat menggunakan mekanisme JSON Web Token (JWT). Token yang dibangkitkan akan disimpan ke dalam komponen Authenticator Database. Komponen berjalan pada port 8084 dengan *endpoint*:

- POST /v1/token
- POST /v1/token/refresh
- GET /.well-known/jwks.json
- GET /health

Request token oleh *client* (POST /v1/token) harus menggunakan mekanisme HTTP Basic Authentication, dengan memberikan *header field* Authorization dengan format

Authorization: Basic <base64(client_id:client_secret)>.

Request untuk *refresh token* (POST /v1/token/refresh) harus disertai *header field* Content-Type bernilai application/json dan *body* berisi JSON yang memiliki *field* refresh_token dan nilai berupa *refresh* token yang diberikan.

Agar Client-facing API mengetahui bahwa JWT yang diberikan oleh *client* kepadanya benar, authenticator menyediakan *endpoint* GET /.well-known/jwks.json yang memberikan JSON Web Key Set (JWKS), yaitu kumpulan *public key* yang bisa digunakan untuk memverifikasi *signature* pada JWT.

6. Authenticator Database

Komponen ini mendukung komponen authenticator dengan menyimpan token yang dibangkitkan oleh authenticator beserta informasi-informasi yang diperlukan authenticator lainnya. Komponen penyimpanan ini sepenuhnya dimiliki oleh komponen authenticator dengan akses komponen dibatasi sehingga hanya komponen authenticator saja yang dapat mengaksesnya, yaitu melalui network internal bernama auth-database dengan komponen berjalan pada port 5432.

7. Client-Facing API

Komponen ini menjadi komponen yang berinteraksi langsung dengan *client* sehingga *client* dapat meminta dan mendapatkan data HazardEvent dari sistem. Komponen ini bertugas memvalidasi JWT, meneruskan *request* ke aggregator, dan membatasi *field* yang diberikan ke *client* sesuai hak aksesnya. Komponen ini berjalan pada port 8080 dengan *endpoint*:

- GET /events
- GET /events/{id}
- GET /source-status
- GET /health

Permintaan data HazardEvent (GET /events dan GET /events/{id}) harus disertai *header field* Authorization yang bernilai JWT yang dibangkitkan oleh komponen authenticator.

8. **Message Broker (Kafka)**

   Message broker dalam sistem ini digunakan untuk meneruskan publikasi *event* oleh aggregator kepada *consumer* yang berlangganan, dalam hal ini dashboard consumer dan notifier consumer. Pada sistem ini, komponen message broker diimplementasikan dengan menggunakan Kafka dan berjalan pada port 9092. Kafka tersebut akan memiliki topik bernama hazard.events.v1 sehingga *consumer* yang berlangganan dengan topik tersebut akan menerima *event* yang dipublikasi oleh aggregator. Terdapat dua *consumer group* yang berlangganan dengan topik tersebut, yaitu dashboard-updater-v1 (berisi komponen dashboard consumer) dan field-notifier-v1 (berisi komponen notifier consumer). Komunikasi komponen ini menggunakan protokol yang disediakan oleh Kafka.

9. **Dashboard Consumer**

   Komponen dashboard consumer membaca *event* dari Kafka pada topik hazard.events.v1 menggunakan *consumer group* dashboard-updater-v1. Komponen ini berjalan pada port 8085 dengan *endpoint*:

   - GET /health
   - GET /events
   - GET /stats

   Komponen ini memiliki basis data lokal tersendiri menggunakan BoltDB pada *file* dashboard.db untuk menyimpan *event* yang didapatnya. Setelah *event* berhasil diterima dan berhasil diproses, dashboard consumer akan meng-*commit offset* Kafka baru sebagai penanda kepada Kafka.

10. **Notifier Consumer**

    Mirip seperti dashboard consumer, komponen notifier consumer membaca *event* dari Kafka pada topik hazard.events.v1 menggunakan *consumer group* field-notifier-v1. Komponen ini berjalan pada port 8086 dengan *endpoint*:

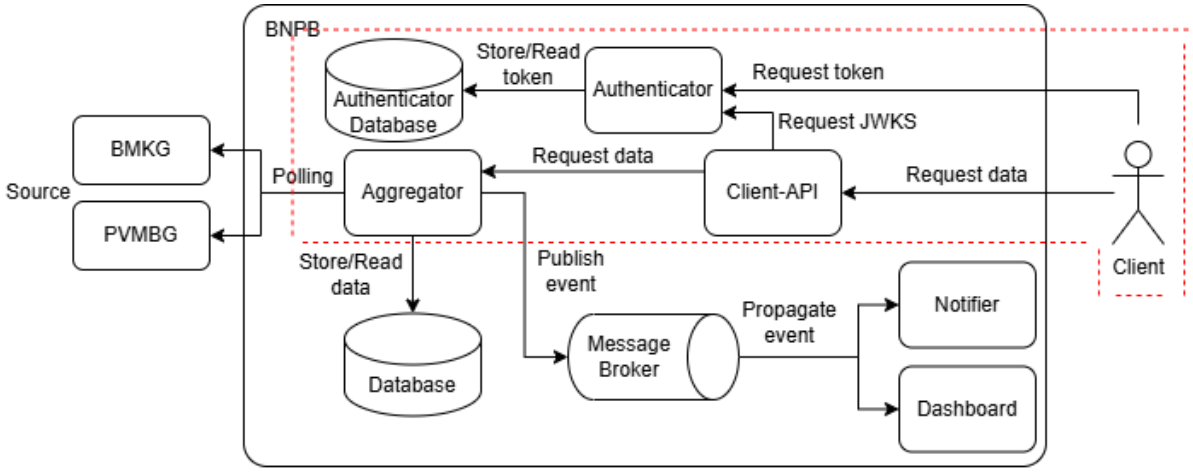
    - GET /health
    - GET /notifications
    - GET /stats

Komponen ini memiliki basis data lokal tersendiri menggunakan BoltDB pada *file* notifier.db untuk menyimpan *event* yang didapatnya. Setelah *event* berhasil diterima dan berhasil diproses, notifier consumer juga akan meng-*commit offset* Kafka baru sebagai penanda kepada Kafka

Alur *Event*

Dalam sistem yang dibuat, terdapat tiga alur utama yang dapat terjadi, yaitu sebagai berikut.

Alur 1: *Client* Meminta Data HazardEvent kepada Sistem



Gambar 2. Komponen Terlibat pada Alur *Event* 1

1. Client -> Authenticator

   Sebelum *client* dapat meminta data HazardEvent kepada sistem, *client* harus memperoleh terlebih dahulu token yang digunakan sebagai kredensial *client* dalam pengaksesan data yang dimiliki sistem. Komponen authenticator menyediakan token kepada *client* dengan cakupan sesuai dengan hak akses *client*, dengan komponen authenticator berjalan pada port 8084 dan menyediakan *endpoint*:

   - POST /v1/token
   - POST /v1/token/refresh

   Saat melakukan *request* token, *client* harus menggunakan HTTP Basic Authentication dengan menambahkan *header field* bernama Authorization pada *header* seperti format berikut.

Authorization: Basic <base64(client_id:client_secret)>

2. Authenticator -> Authenticator Database

   Ketika komponen authenticator menerima *request* token dari *client*, authenticator antara akan membuat token jika belum ada di basis data lalu menyimpannya dalam basis data khusus keperluan authenticator atau membaca token yang sudah ada di basis data. Komponen authenticator database diakses langsung oleh komponen authenticator secara khusus melalui network internal bernama auth-database dan berjalan pada port 5432.

3. Client -> Client-facing API

   Setelah *client* mendapatkan token yang sesuai, *client* dapat melakukan *request* kepada sistem melalui komponen Client-facing API yang berjalan pada port 8080 dengan menyediakan *endpoint*:

   - GET /events
   - GET /events/{id}

   Request yang diberikan oleh *client* harus menyertakan *header field* Authorization yang berisi token yang didapatkan dari authenticator dengan format sebagai berikut.

   Authorization: Bearer <JWT access token>

4. Client-facing API -> Authenticator

   Komponen Client-facing API perlu memvalidasi token yang diberikan *client* kepadanya dengan meminta *public signing key* yang disediakan komponen authenticator melalui *endpoint* (pada port 8084):

   - GET ./well-known/jwks.json

   Authenticator akan memberikan *response* JSON Web Key Set (JWKS): kumpulan *public key* untuk memverifikasi *signature* JWT.

5. Client-facing API -> Aggregator

   Client-facing API akan meneruskan permintaan *client* kepada komponen aggregator ketika *request* dari *client* berhasil melalui proses autentikasi. Komponen aggregator berjalan pada port 8083 dan menyediakan *endpoint*:

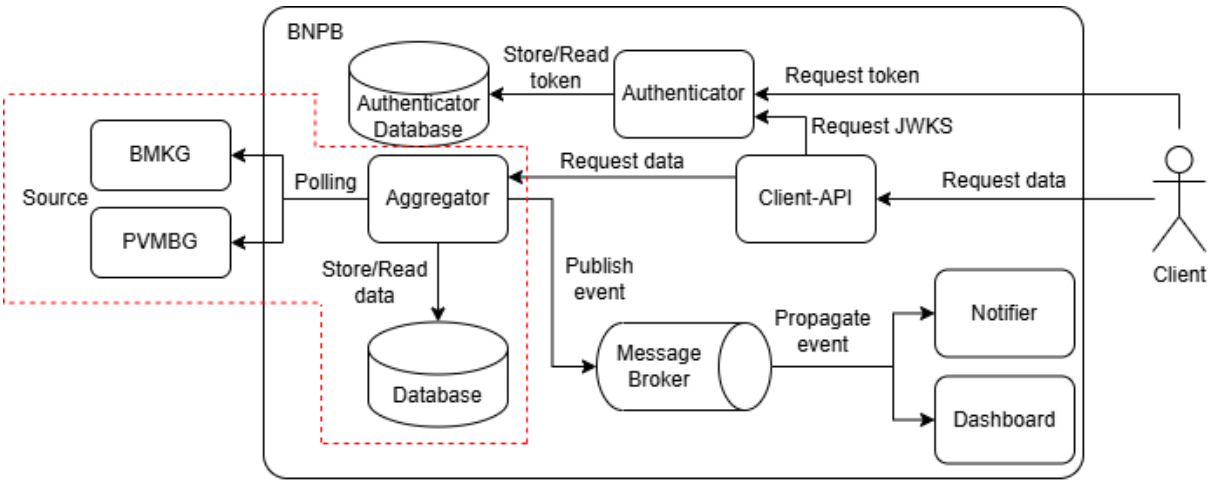
- GET /internal/events
- GET /internal/events/{id}
- GET /internal/source-status

Request yang dibuat kepada aggregator harus menyediakan *header field* X-Aggregator-Key sebagai bentuk autentikasi dengan nilai yang sesuai dengan yang tertera pada .env *file* serta perlu menyediakan *header field* X-Correlation-ID berisi waktu saat *request* dibuat sehingga dimungkinkan untuk melakukan pelacakan permintaan di sistem.

6. Aggregator -> Database

Aggregator menjawab *request* dari Client-facing API dengan mengirimkan data HazardEvent yang didapat dari hasil pembacaan data dari komponen database yang dimilikinya. Komponen database diakses langsung oleh aggregator secara khusus melalui network internal bernama database dan berjalan pada port 5432.

Alur 2: *Polling* dan Agregasi dari Instansi Sumber



Gambar 3. Komponen Terlibat pada Alur *Event* 2

1. Aggregator -> BMKG

Komponen aggregator mengambil data SeismicEvent dan TsunamiWarning secara berkala dari BMKG. Layanan BMKG berjalan pada port 8081 dan menyediakan *endpoint*:

- GET /seismic-events?since=<timestamp>
- GET /tsunami-warnings?since=<timestamp>

Request harus menyertai *header field* X-BMKG-Key yang bernilai sesuai di .env *file* serta *field* X-Correlation-ID sebagai pelacak permintaan. Kedua *request* tersebut masing-masing akan memberikan *response* yaitu data SeismicEvent dan TsunamiWarning yang waktu kemunculannya setelah nilai timestamp pada *query request*. Aggregator akan mem-*parse* data-data yang didapat, memetakannya ke HazardEvent, dan menyimpannya ke dalam *database*.

2. Aggregator -> PVMBG

   Komponen aggregator mengambil data VolcanoReport secara berkala dari PVMBG. Layanan PVMBG berjalan pada port 8082 dan menyediakan *endpoint*:

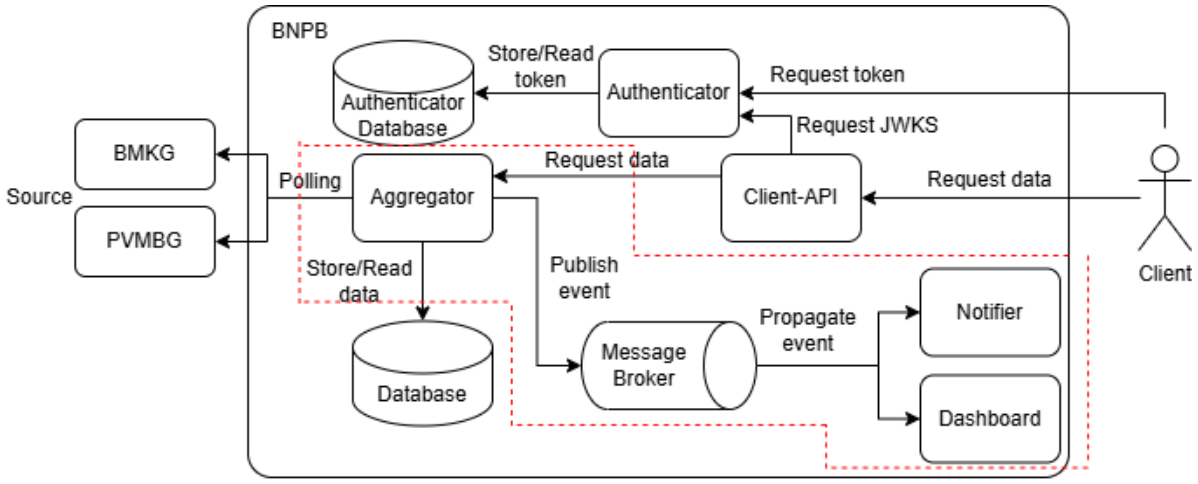
   - GET /volcanic-reports?since=<timestamp>

   Request harus menyertai *header field* Authorization yang berisi token PVMBG serta *field* X-Correlation-ID sebagai pelacak permintaan. Kedua *request* tersebut masing-masing akan memberikan *response* yaitu data SeismicEvent dan TsunamiWarning yang waktu kemunculannya setelah nilai timestamp pada *query request*. Sama seperti yang dilakukan pada data dari BMKG, aggregator akan mem-*parse* data-data yang didapat, memetakannya ke HazardEvent, dan menyimpannya ke dalam *database*.

3. Aggregator -> Database

   Setelah aggregator berhasil memetakan data ke HazardEvent, data hasil pemetaan disimpan ke dalam komponen database yang dimilikinya. Sama seperti proses pembacaan data oleh aggregator, komponen database diakses langsung oleh aggregator secara khusus melalui network internal bernama database dan berjalan pada port 5432.

Alur 3: Publikasi *Event* melalui Message Broker



Gambar 4. Komponen Terlibat pada Alur *Event* 3

1. Aggregator -> Message Broker

   Selain menyimpan HazardEvent, aggregator juga mempublikasi HazardEvent baru yang didapat ke Message Broker berupa Kafka. Publikasi dilakukan dengan *binary protocol* dari Kafka (melalui TCP) pada port 9092. *Topic* tujuan publikasi bernama hazard.events.v1 dengan menggunakan *message key* hazard_id (atribut kunci pada HazardEvent). *Event* yang dipublikasi oleh aggregator berbentuk JSON *envelope* dengan format sebagai berikut.

```
{
  "event_type": "hazard.event.upserted",
  "schema_version": 1,
  "correlation_id": "<correlation_id>",
  "published_at": "<time_publication>",
  "data": {
    "hazard_id": "<HazardEvent.hazard_id>",
    "source": "<BMKG/PVMBG>",
    "source_ref_id": "<HazardEvent.source_ref_id>",
    "hazard_type": "<HazardEvent.hazard_type>",
    "severity": "<HazardEvent.severity>",
    "attributes": <HazardEvent.attributes>
  }
}
```

2. Message Broker -> Dashboard Consumer & Notifier Consumer

Komponen Dashboard Consumer dan Notifier Consumer berlangganan pada topik hazard.events.v1 yang disediakan message broker dengan menggunakan *consumer group* masing-masing bernama dashboard-updater-v1 dan field-notifier-v1 sehingga keduanya akan menerima *event* yang dipublikasikan pada topik tersebut secara independen. Komunikasi message broker kepada dashboard consumer dan notifier consumer dilakukan melalui protokol dari Kafka. Dashboard consumer dan notifier consumer akan melakukan *fetch*, men-*decode* JSON *envelope* yang didapat untuk mendapatkan data HazardEvent, dan menyimpan data tersebut pada basis data lokal miliknya (menggunakan BoltDB di *file* masing-masing bernama dashboard.db dan notifier.db). Dashboard serta notifier lalu merespon komponen message broker dengan meng-*commit* offset Kafka yang baru sebagai penanda bahwa *event* yang di-*fetch* telah berhasil diterima.

Batas Kontainer

Berikut adalah daftar *container* yang terdefinisi pada sistem.

Nama Container	Komponen Terkait	Port
bmkg	BMKG	8081
pvmbg	PVMBG	8082
aggregator	Aggregator	8083
postgres	Database	5432 melalui *network* internal database
kafka	Message Broker	9092
auth	Authenticator	8084
auth-postgres	Authenticator Database	5432 melalui *network* internal auth-database
client-api	Client-facing API	8080
dashboard-consumer	Dashboard Consumer	8085
notifier-consumer	Notifier Consumer	8086

Teknologi yang Digunakan

Secara keseluruhan, berikut *tech stack* yang digunakan dalam implementasi sistemnya

Service	**Teknologi**
BMKG & PVMBG mock server	Golang (Chi)
Aggregator	Golang, PostgreSQL, Kafka
Auth & Client-Facing API	Golang, PostgreSQL
Dashboard & notifier consumer	Golang, Kafka, bbolt
Microservice & containerization	Docker, kafka

1. **Bahasa Pemrograman**

   Bahasa yang digunakan dalam membuat arsitektur seluruh sistem ini yakni Golang. Golang digunakan karena mekanisme *concurrency*-nya yang sudah sangat baik melalui *goroutine* beserta *library support* yang sangat luas. Selain itu, secara sintaks Golang termasuk salah satu bahasa pemrograman yang intuitif, *verbose*, dan mudah untuk digunakan. Dengan *balance* dari performa dan konkurensi yang sudah baik beserta sintaks yang mudah dipahami membuat Golang menjadi pilihan kami untuk mengimplementasikan arsitektur sistem ini.

2. **Database**

   Pilihan *database* untuk sistem pada kasus arsitektur yang mementingkan suatu data terformat dengan ketat dengan *field* yang teratur membuat kita memilih PostgreSQL karena properti-properti tersebut. Selain itu, PostgreSQL mempunyai keuntungan di mana merupakan salah satu *database* yang sangat mendukung skalabilitas sehingga ketika sewaktu-waktu database akan dipindahkan ke server terpisah, PostgreSQL tidak akan sulit untuk mengadaptasi dengan arsitektur sistem sekarang.

3. **Message Broker**

   Kami memilih Apache Kafka sebagai *message broker* dikarenakan model event yang menurut kami lebih sesuai untuk memakai Kafka. Karena Kafka memungkinkan

suatu dashboard dapat membuat publish tanpa bergantung suatu consumer terlebih dahulu, pola ini menjadi alasan dari kami untuk menggunakan Kafka karena sistem tiap consumer dapat diperlakukan secara independen.

Asumsi

Petunjuk Menjalankan Sistem

Salin file dari .env.example ke .env dan isi konfigurasi environment variables sesuai dengan fieldnya masing-masing.

```
cp .env.example .env
```

Setelah semua variable terisi, sistem dapat dijalankan langsung dengan menjalankan docker compose, di mana compose tersebut akan memulai semua mock server dan konfigurasi tiap servicenya

```
docker compose up --build -d
```

Untuk *command* selengkapnya (pengujian), penulis juga menyediakan README pada *repository* yang bisa dijadikan sebagai acuan dalam mengoperasikan *service*.

Penyelesaian Problem

Problem 1: Arsitektur & Interoperabilitas Data

Masalah pertama menyatakan bahwa instansi sumber dapat sewaktu-waktu mengubah skema yang dikirim kepada BNPB tanpa koordinasi sebelumnya sehingga jika kemungkinan tersebut tidak ditangani bisa mengakibatkan perilaku tidak terduga pada pihak BNPB.

Rancangan Solusi

Solusi yang dipilih untuk menyelesaikan permasalahan tersebut adalah memasukkan *field* baru pada skema (tidak ada dalam kontrak awal) ke dalam atribut bernama attributes bersama dengan *field-field* lainnya yang tidak dipetakan langsung ke atribut spesifik pada HazardEvent, dengan asumsi instansi sumber hanya melakukan perubahan skema dengan menambah *field* baru saja (tidak menghapus atau mengubah *field* yang sudah ada dalam kontrak) melalui mekanisme JSON *tolerant parsing* dengan kebijakan *additive-only* dan *dynamic field mapping*. Hal tersebut dicapai dengan mendesain struktur data HazardEvent (pada komponen aggregator) agar atribut bernama attributes memiliki tipe JSON sehingga aggregator bebas memasukkan *field* tambahan dengan format *key-value*. Proses *parsing* pada aggregator juga perlu dibuat sehingga bersifat JSON *tolerant*, memasukkan *key-value* yang tidak diketahui ke dalam atribut Extra (bertipe JSON). Dengan kedua hal tersebut, tercapailah mekanisme *dynamic field mapping* pada aggregator terhadap data dari instansi sumber (SeismicEvent bersama TsunamiWarning dan VolcanicReport) ke struktur data HazardEvent.

Alasan Keputusan

Beberapa alternatif solusi berikut telah dipertimbangkan.

No.	Alternatif solusi	Kelebihan	Kekurangan
1	Hanya *field* yang terdapat di kontrak awal dipetakan ke HazardEvent, selain itu dibuang.	Proses *mapping* data ke HazardEvent menjadi sederhana karena tidak perlu memikirkan *field* baru sehingga dapat di-*hardcode*.	Data yang ada di *field* baru akan hilang begitu saja, padahal bisa jadi data tersebut penting bagi *event consumer*.
2	*Field* baru dimasukkan ke dalam atribut	Data yang ada di *field* baru tetap terekam ke dalam	*Mapping* data ke HazardEvent menjadi lebih kompleks

	attributes sebagai *key-value* dalam JSON.	HazardEvent sehingga tidak ada kehilangan informasi ketika terjadi penambahan *field* pada skema.	karena harus menyimpan terlebih dahulu *field* yang tidak diketahui ke dalam JSON lalu memetakannya ke atribut attributes pada HazardEvent.
3	Semua *field* dianggap tidak diketahui oleh aggregator, data disimpan dalam struktur JSON sepenuhnya.	Memungkinkan instansi sumber untuk memodifikasi skema data yang dikirim sebebas-bebasnya (bahkan dapat menghapus atau mengubah *field* yang sudah ada sebelumnya) tanpa terjadi kehilangan data atau keperluan untuk modifikasi skema pada komponen aggregator.	Data menjadi sulit diolah baik bagi sistem BNPB (dalam menentukan mana *field* mentah dan ringkasan untuk dikirim ke *client* dengan hak akses tertentu) maupun *client* (manual mem-*parsing* data berbentuk JSON yang tidak memiliki struktur tetap).

Dari ketiga alternatif tersebut, dipilih alternatif solusi nomor dua yang memberikan fleksibilitas skema data dari instansi sumber tanpa mengorbankan kompleksitas tambahan dalam menangani struktur data JSON yang tidak tetap.

Batasan yang Tidak Ditangani

Dalam perancangan solusi permasalahan ini, terdapat hal yang tidak ditangani, yaitu sebagai berikut.

- Pengubahan nama field yang sudah ada di kontrak awal (contoh, mengubah field "region_name" menjadi "region" atau "occured_at" menjadi "time_occured").
- Penghapusan field yang sudah ada di kontrak awal.
- Pengubahan tipe data field (contoh, field depth_km yang awalnya bertipe float diubah menjadi tipe integer).

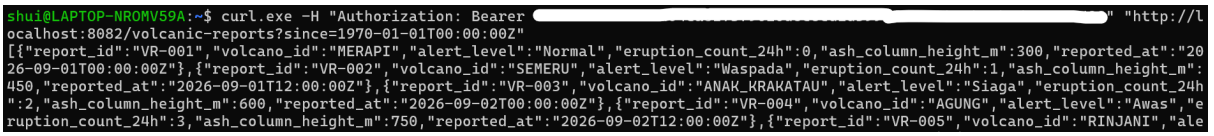
Alasan hal-hal tersebut tidak ditangani oleh solusi yang diimplementasi adalah karena, pada permasalahan, narasi hanya menyebutkan masalah penambahan *field* pada data dari instansi sumber serta ringkasan masalah menyebutkan masalah yang dihadapi adalah terjadinya skema aditif saja oleh instansi sumber.

Bukti Penyelesaian

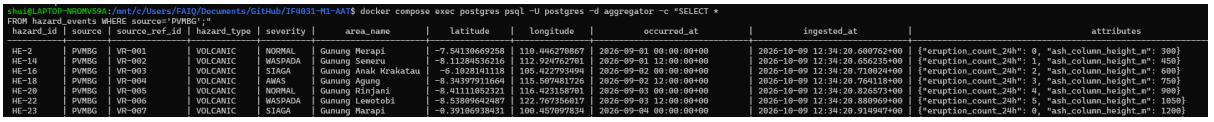
Berikut adalah bukti penyelesaian permasalahan pertama.

1. **PVMBG pada skema awal, aggregator menghasilkan HazardEvent yang valid**

   Command: curl.exe -H "Authorization: Bearer <PVMBG Token>" "http://localhost:8082/volcanic-reports?since=1970-01-01T00:00:00Z"

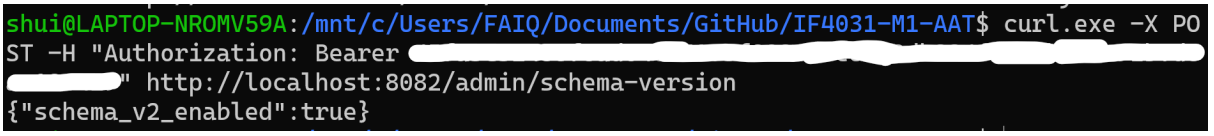


   Command: docker compose exec postgres psql -U postgres -d aggregator -c "SELECT * FROM hazard_events WHERE source='PVMBG';"

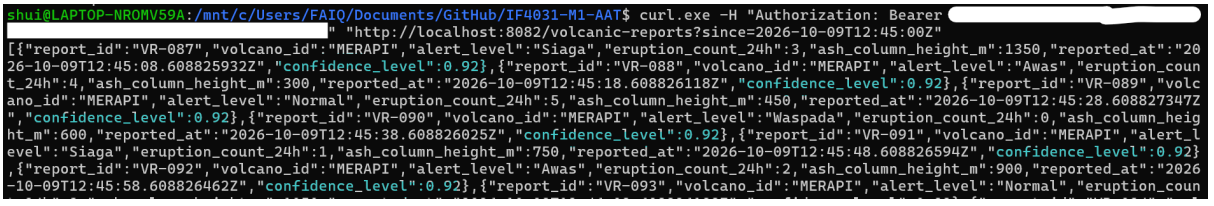


2. **Perubahan skema PVMBG tanpa re-*deploy***

   Command: curl.exe -X POST -H "Authorization: Bearer <PVMBG Token>" http://localhost:8082/admin/schema-version

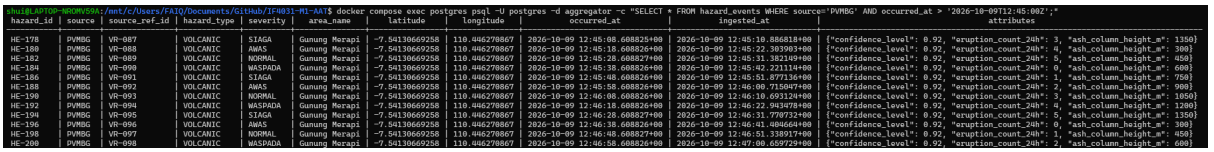


   Command: curl.exe -H "Authorization: Bearer <PVMBG Token>" "http://localhost:8082/volcanic-reports?since=<Waktu UTC Pengubahan Skema>"



3. **Aggregator tetap menghasilkan HazardEvent yang valid**

   Command: docker compose exec postgres psql -U postgres -d aggregator -c "SELECT * FROM hazard_events WHERE source='PVMBG' AND occured_at > <Waktu UTC Pengubahan Skema>;"



Jawaban terhadap Butir Permasalahan

Jawaban atas butir dari permasalahan yang perlu dijelaskan di laporan adalah sebagai berikut.

- (Butir 4) *Field* yang tidak dikenal akan ditangani dengan meneruskannya ke *field* bernama attributes pada HazardEvent. Perilaku tersebut dapat dilihat pada gambar yang ada di bukti penyelesaian nomor 3, terlihat bahwa terdapat *key-value* baru dengan *key* bernilai confidence_level.

- (Butir 5) Aggregator didesain menggunakan mekanisme dynamic field mapping untuk memetakan field yang tidak dikenal ke dalam field bernama attributes pada HazardEvent dengan kebijakan *additive-only* yang artinya perubahan skema yang didukung hanya penambahan *field* dari instansi sumber dan tidak mendukung perubahan/penghapusan *field*.

Problem 2: *Concurrency, Availability, & Graceful Degradation*

Problem 2 membahas tiga masalah utama pada gateway BNPB. Pertama, *head-of-line blocking* terjadi ketika permintaan data BMKG ikut tertahan oleh pemanggilan PVMBG yang lebih lambat. Kedua, *resource exhaustion* yang dapat terjadi saat lonjakan akses karena jumlah permintaan bersamaan tidak dibatasi. Ketiga, ketiadaan degradasi terkendali menyebabkan gangguan PVMBG berdampak pada seluruh layanan, meskipun data BMKG masih tersedia dan data PVMBG sebelumnya telah tersimpan. Oleh karena itu, solusi perlu menjaga agar layanan BMKG tetap responsif, membatasi beban request, dan tetap memberi respons yang berguna atau status ketidaktersediaan yang jelas ketika layanan mengalami *outage*.

Rancangan Solusi

Solusi yang dipilih untuk menyelesaikan permasalahan tersebut adalah Aggregator menjalankan polling BMKG dan PVMBG secara terpisah menggunakan dua goroutine. Keduanya berjalan di latar belakang dan menyimpan hasil yang berhasil diperoleh ke Canonical Store. Dengan demikian, permintaan pembacaan data dari client mengambil data tersimpan melalui API dan tidak perlu menunggu polling BMKG atau PVMBG selesai. Snippet kode implementasi solusi rancangan tersebut dapat dilihan pada fungsi Run() pada aggregator.

```
func (a *Aggregator) Run(ctx context.Context) {
    go a.runBMKG(ctx)
    go a.runPVMBG(ctx)
    <-ctx.Done()
}
```

Untuk membatasi penggunaan resource, Client-Facing API dan endpoint internal Aggregator masing-masing membatasi jumlah permintaan aktif. Di mana nilai bawaan batasnya permintaan serentak per service diatur di variable .env pada field CLIENT_API_MAX_IN_FLIGHT serta AGGREGATOR_MAX_IN_FLIGHT (pada implementasi kami, secara nilai *default* kami set menjadi 64). Jika seluruh slot sedang digunakan, permintaan baru ditolak dengan HTTP 429 dan header Retry-After: 1.

Selain itu, aggregator juga mencatat status tiap sumber untuk memastikan ketika suatu *outage* terjadi pada layanan PVMBG dan BMKG agar setidaknya sebagian data masih tersedia untuk pengguna. Setelah *polling* berhasil, status sumber menjadi UP. Jika polling gagal setelah sebelumnya berhasil, statusnya menjadi STALE dan respons menyertakan waktu sejak kegagalan pertama (stale_since). Jika belum pernah berhasil, statusnya menjadi UNAVAILABLE. Data yang sudah ada di Canonical Store tetap dapat dibaca selama sumbernya mengalami *outage*. Setelah polling kembali berhasil, status berubah menjadi UP tanpa perlu me-*restart* sistem.

```
func (t *Tracker) Success(source string, now time.Time) {
    t.mu.Lock()
    defer t.mu.Unlock()

    now = now.UTC()
    t.sources[source] = SourceStatus{State: "UP", LastSuccessAt:
&now}
}

func (t *Tracker) Failure(source string, now time.Time) {
    t.mu.Lock()
    defer t.mu.Unlock()

    current := t.sources[source]
    if current.StaleSince == nil {
        now = now.UTC()
        current.StaleSince = &now
    }
    if current.LastSuccessAt == nil {
        current.State = "UNAVAILABLE"
    } else {
        current.State = "STALE"
    }
    t.sources[source] = current
}
```

Alasan Keputusan

Beberapa alternatif solusi telah dipertimbangkan sebagai berikut.

No.	Alternatif solusi	Kelebihan	Kekurangan
1	Mengambil data dari BMKG dan PVMBG secara berurutan saat ada permintaan *client*.	Alurnya sederhana karena data langsung diambil dari sumber ketika diminta.	Sumber yang lambat dapat menahan permintaan yang membutuhkan sumber lain. Selain itu, gangguan sumber juga berisiko membuat seluruh permintaan gagal atau menunggu hingga *timeout*.
2	Menjalankan seluruh permintaan secara serentak tanpa batas.	Permintaan tidak perlu menunggu slot untuk mulai diproses.	Mengharuskan suatu sistem mempunyai resource CPU beserta *memory* yang mencukupi, dikarenakan lonjakan akses yang besar dapat menghabiskan koneksi, memori, atau *resource* lainnya.
3	Melakukan polling tiap sumber secara independen, membaca data dari Canonical Store, dan membatasi permintaan aktif dengan HTTP 429 saat kapasitas tercapai.	Kelambatan satu sumber tidak menahan pembacaan data dari sumber lain.	Implementasi yang lebih kompleks karena menerapkan konsep *polling*. Selain itu, implementasi tidak menjamin data paling terbaru karena data untuk sumber yang *outage* merupakan data lama sampai *polling* berhasil kembali.

Dari ketiga alternatif tersebut, dipilih alternatif solusi nomor tiga. Model ini mengurangi ketergantungan antara sumber data, permintaan client, dan kondisi outage, sambil menjaga jumlah pekerjaan aktif tetap terbatas sehingga tidak memerlukan kapabilitas resource yang terlalu besar.

Batasan yang Tidak Ditangani

Dalam perancangan solusi permasalahan ini, terdapat hal yang tidak ditangani, yaitu sebagai berikut.

- Ketika batas slot aktif sudah mencapai batas (dalam kasus implementasi *default* yakni 64) gateway akan me-*return* HTTP 429 yang dimana hal tersebut merupakan penolakan yang disengaja saat kapasitas penuh. Dari sisi *client*, *client* perlu menangani respons tersebut (contohnya bisa dapat mencoba request kembali berdasarkan header Retry-After).

- Data berstatus STALE tetap disajikan dari Canonical Store sehingga *client* perlu memeriksa status sumber dan stale_since sebelum menggunakan data lama untuk mengambil keputusan.

Bukti Penyelesaian

Berikut adalah bukti penyelesaian permasalahan.

1. **Head-of-line blocking**

   Sebelum mengetes, kira memerlukan untuk mengubah default konfigurasi PVMBG agar mensimulasikan kasus *service* PVMBG yang sedang lambat dari

   ```
   PVMBG_DELAY_MIN_MS=500
   PVMBG_DELAY_MAX_MS=3000
   ```

   Menjadi

   ```
   PVMBG_DELAY_MIN_MS=6000
   PVMBG_DELAY_MAX_MS=6000
   ```

   Dengan konfigurasi delay ini, kita coba mengetes endpoint PVMBG dan BMKG

   ```
   Thertaselette@DESKTOP-G3N13UL:~/Kuliah/Semester-5/AAT/IF4031-M1-AAT$ time curl --max-time
    10 -fsS -H "Authorization: Bearer $TOKEN" "http://localhost:8080/events?source=PVMBG" |
   jq '{event_count: (.events | length), source: .sources.PVMBG}'
   {
     "event_count": 50,
     "source": {
       "state": "STALE",
       "last_success_at": "2026-10-09T08:37:14.023542637Z",
       "stale_since": "2026-10-09T08:37:19.151026679Z"
     }
   }

   real    0m0.010s
   user    0m0.008s
   sys     0m0.001s
   Thertaselette@DESKTOP-G3N13UL:~/Kuliah/Semester-5/AAT/IF4031-M1-AAT$ time curl --max-time 10 -fsS -H "Authorization: Bearer $TOKEN" "http://localhost:8080/events?
   source=BMKG" | jq '{event_count: (.events | length), source: .sources.BMKG}'
   {
     "event_count": 50,
     "source": {
       "state": "UP",
       "last_success_at": "2026-10-09T08:38:37.71390387Z"
     }
   }

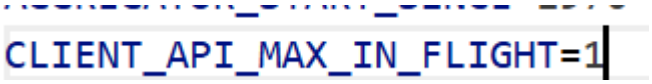
   real    0m0.009s
   user    0m0.002s
   sys     0m0.007s
   ```

   Dapat terlihat bahwa pada kasus PVMBG terjadi jeda antara last_success_at (~08:37:14) dan stale_since (~08:37:19) karena terdapat *delay* pada PVMBG itu sendiri, di mana PVMBG tidak mampu memberikan data terbarunya, alhasil *service*

memberikan label bahwa PVMBG sedang “STALE” dengan mengembalikan data PVMBG hasil dari polling sebelumnya (bukan data terbaru). Di sisi lain, *service* BMKG masih berjalan dengan normal yang dapat terlihat dari kondisinya yang selalu “UP”.

2. **Resource exhaustion**

Sebelum mengetes, kira memerlukan untuk mengubah default konfigurasi CLIENT_API_MAX_IN_FLIGHT menjadi 1 untuk mensimulasikan hanya ada satu resource tempat disediakan saja untuk mengetes cara kerja resource exhaustion



Cara mengetes resource exhaustionnya dapat kita lakukan dengan langsung menembak salah satu *service* berkali-kali sehingga *service* terpaksa melakukan return HTTP 429 ketika tempat service saat ini sedang penuh



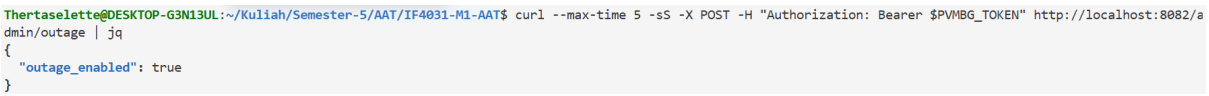
Dapat dilihat dari gambar di atas, kita menembak 20 kali pada BMKG *service* di mana responsenya menunjukkan bahwa sebagian hasil akan ditolak (429) karena saat ini kapasitas layanan *service* sudah mencapai batas (yakni 1), sebagian juga akan diterima (200) karena saat kondisi pada waktu tersebut kebetulan dari *service* memiliki tempat kosong untuk menerima suatu *request* yang baru.

3. **Ketiadaan degradasi terkendali**

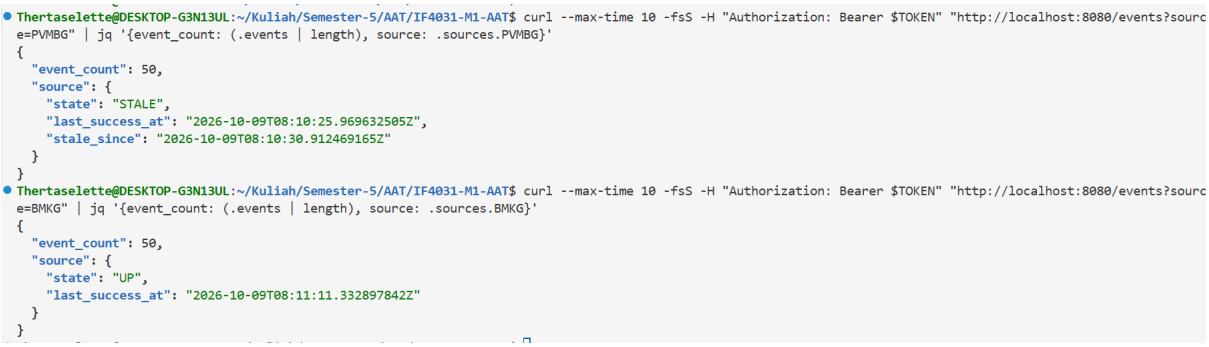
Diberikan konfigurasi PVMBG & BMKG normal



Kita bisa mengaktifkan /admin/outage pada PVMBG dengan harapan dimana PVMBG berubah statusnya menjadi down. Namun, dengan konfigurasi *polling* yang sudah dibuat, kita dapat membuat PVMBG berstatus STALE, dengan memastikan status BMKG masih tetap UP



Ketika mengecek konfigurasi setelah /admin/outage



Telah dibuktikan implementasi berhasil, dengan user secara teori masih dapat mengambil data di PVMBG selama ada data di Canonical Store, dengan BMKG user dapat mengambil secara normal

Jawaban terhadap Butir Permasalahan

1. **Head-of-line blocking**

Dilakukan sebuah *polling* BMKG dan PVMBG yang dijalankan secara independen dengan client membaca data yang sudah tersimpan di Canonical Store. Karena itu, permintaan yang hanya memerlukan data BMKG tidak perlu menunggu polling PVMBG.

2. **Resource exhaustion**

Membatasi request Client-Facing API dan Aggregator dengan suatu set request yang terbatas. Jika seluruh slot pada suatu service sedang terpakai, permintaan berikutnya ditolak dengan HTTP 429 dan header Retry-After: 1.

3. **Ketiadaan degradasi terkendali**

Ketika PVMBG mengalami outage, data PVMBG yang sudah tersimpan tetap dapat dibaca melalui API. Jika PVMBG belum pernah berhasil diakses, statusnya UNAVAILABLE, tetapi jika bukan respons menyertakan status sumber STALE beserta stale_since. Dengan ini,meskipun *outage* terjadi, Data seismik BMKG tetap dapat dilayani hingga, status layanan menjadi UP kembali.

Problem 3: *Service-to-Service Authentication & Trust*

Problem 3 membahas tentang tidak adanya pemisahan domain kepercayaan (*trust domain*) antara BMKG, PVMBG, layanan internal BNPB, dan *client downstream*. Jika seluruh pihak menggunakan satu kredensial yang sama, kebocoran kredensial Media dapat membuka akses ke *raw data* maupun layanan *instance* sumber. Oleh karena itu, solusi perlu memastikan bahwa setiap domain menggunakan identitas yang berbeda, akses data dibatasi berdasarkan *scope*, dan pembatasan tersebut di-*enforce* oleh server.

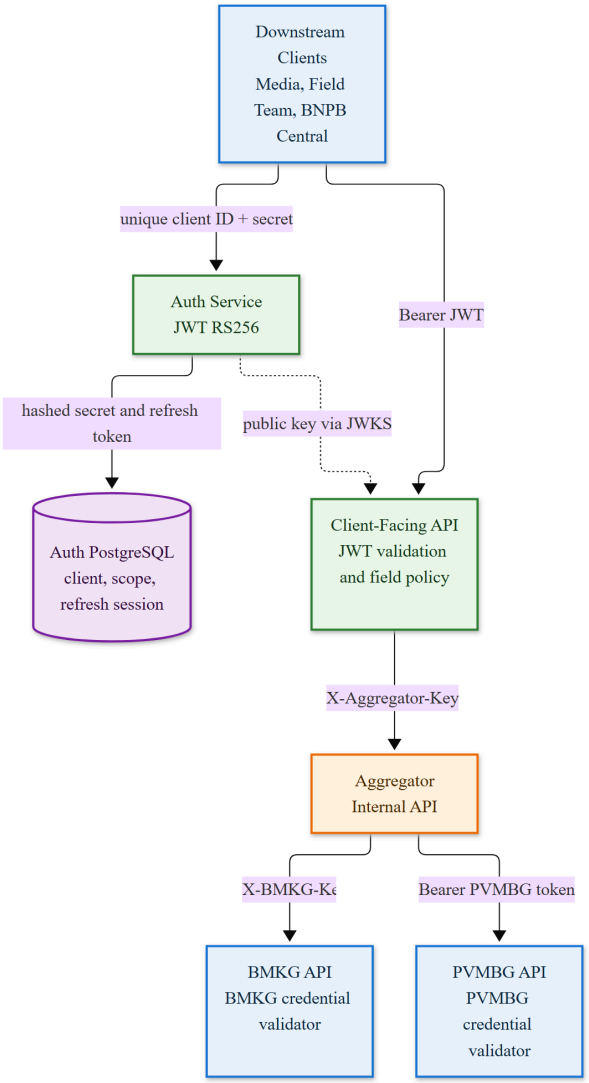
Rancangan Solusi

Solusi yang dipilih adalah memisahkan kredensial berdasarkan hubungan antarlayanan. Aggregator menggunakan API *key* khusus ketika mengakses BMKG dan token khusus ketika mengakses PVMBG. Client-Facing API juga menggunakan API *key* tersendiri untuk membaca Canonical Store melalui API Aggregator. Dengan demikian, tidak ada satu kredensial yang dapat digunakan untuk mengakses seluruh layanan.

Domain	Mekanisme Autentikasi
Aggregator -> BMKG	API *key* melalui *header* “X-BMKG-Key”
Aggregator -> PVMBG	Token melalui *header* “Authorization: Bearer <PVMBG_TOKEN>”
Client-Facing API -> Aggregator	API *key* melalui *header* “X-Aggregator-Key”
Client downstream -> Auth Service	Client ID dan *client secret* melalui HTTP
Client downstream -> Client-Facing API	JWT *access token* melalui skema Bearer
Auth Service -> Client-Facing API	RSA *signature* yang di-*verify* menggunakan *public key* dari JWKS

Pembagian tanggung jawab keamanan setiap komponen adalah sebagai berikut.

Komponen	Tanggung Jawab
BMKG dan PVMBG	Memvalidasi kredensial *instance*-nya sendiri dan menolak kredensial yang tidak sesuai
Auth Service	Melakukan validasi pada identitas *client*, menentukan *scope* pada sisi server, menerbitkan JWT, dan *rotate* token
Auth PostgreSQL	Menyimpan *hash client secret* serta status *refresh session* dan hanya dapat diakses oleh Auth Service
Client-Facing API	Memvalidasi JWT menggunakan JWKS dan menerapkan kebijakan akses *field* sebelum data dikirim kepada *client*
Aggregator	Memvalidasi API *key* internal dari Client-Facing API, mengakses BMKG dan PVMBG dengan kredensial berbeda, serta menjadi pemilik Canonical Store

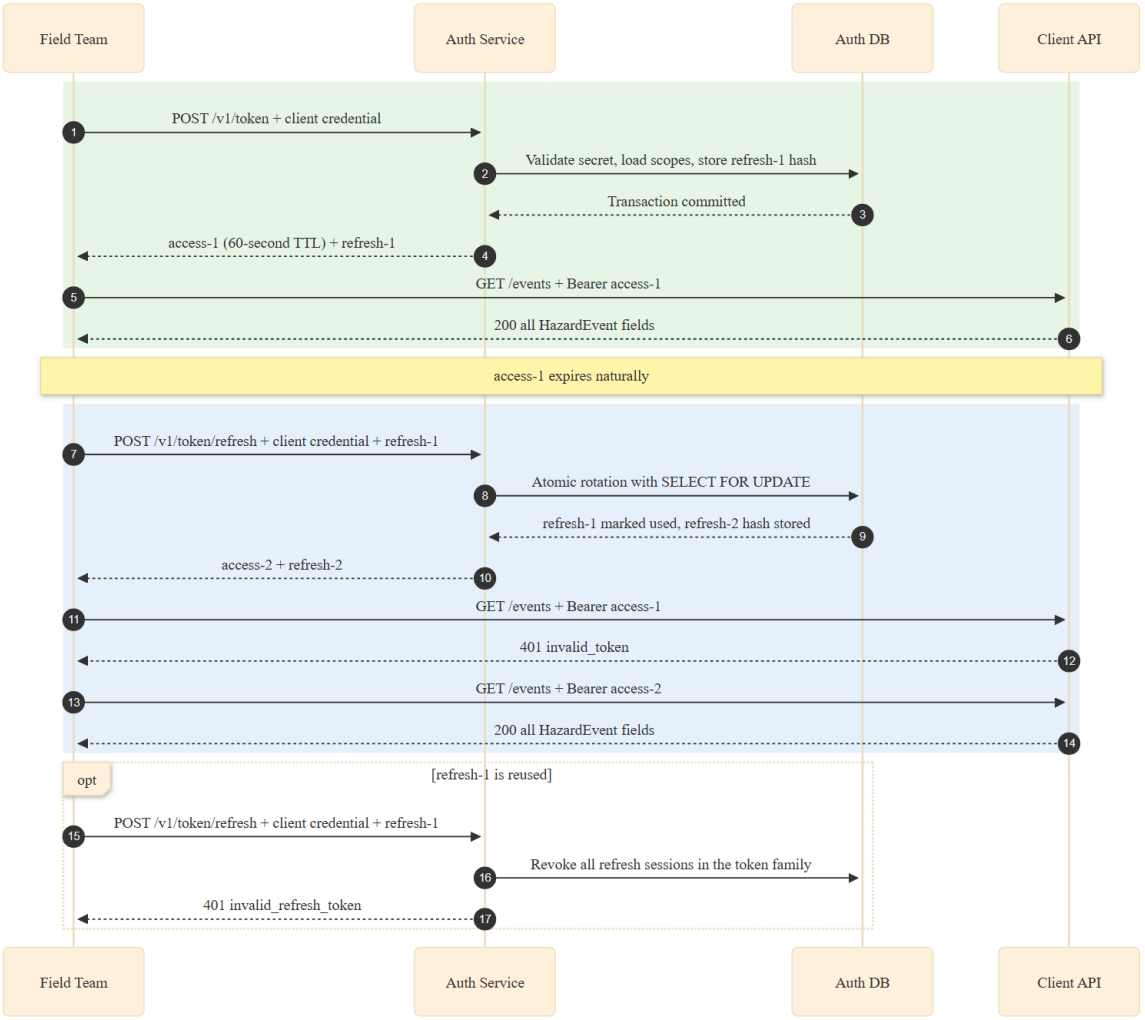


Gambar 5. Pemisahan *trust domain*

Alur autentikasi dan otorisasi dimulai ketika *client* mengirim client ID dan *client secret* ke *endpoint* “POST /v1/token”. Media, Tim Lapangan, dan BNPB Pusat mempunyai identitas dan *secret* yang berbeda. Auth Service memvalidasi *hash secret*, mengambil *scope* yang sudah ditentukan pada sisi server, lalu menerbitkan *access token* dan *refresh token*. Media hanya memperoleh “hazards:read:summary”, sedangkan Tim Lapangan dan BNPB Pusat memperoleh “hazards:read:summary” serta “hazards:read:raw”. Karena *scope* tidak diterima dari input *client*, Media tidak dapat menaikkan *privilege*-nya dengan meminta *scope* mentah.

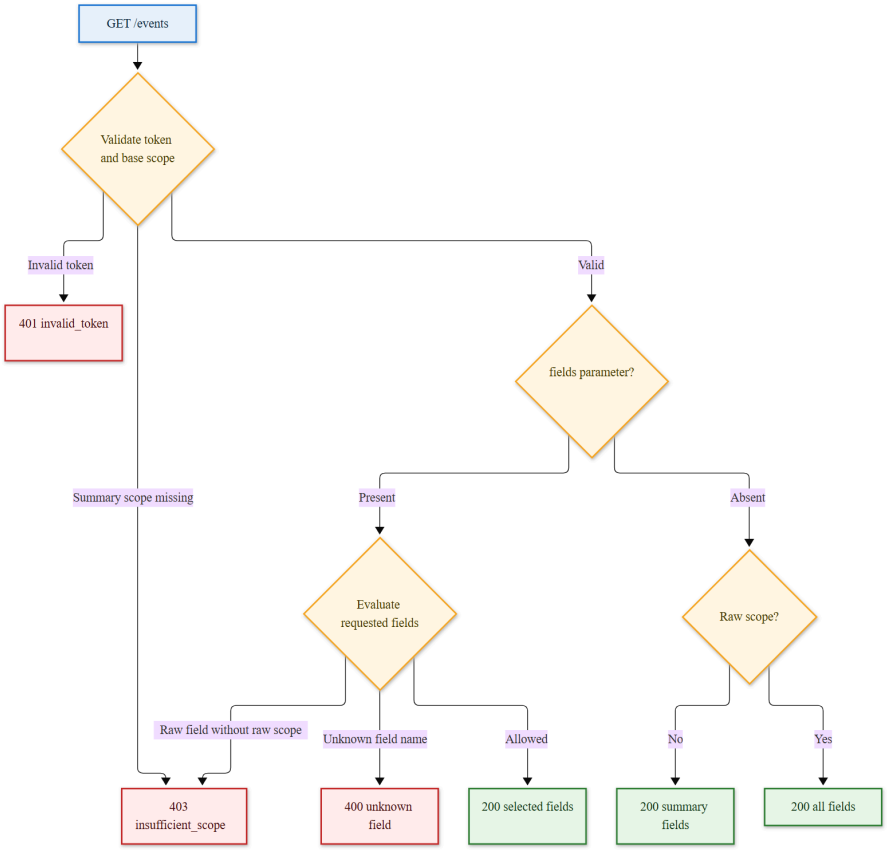
Selanjutnya, *Access token* dibentuk dengan JWT yang ditandatangani dengan algoritma RS256 dan memiliki TTL (*Time To Live*) *default* 60 detik. Token memuat identitas *client*, *scope*, identitas sesi, *issuer*, *audience*, waktu *publish*, dan waktu kedaluwarsa. *Private key* hanya dimiliki Auth Service, sedangkan Client-Facing API mengambil dan menyimpan sementara *public key* dari *endpoint* "/.well-known/jwks.json". Pada setiap permintaan data, Client-Facing API memvalidasi algoritma RS256, tipe token, "kid", *signature*, *issuer*, *audience*, identitas, waktu kedaluwarsa, dan *scope*. Token yang tidak lolos pemeriksaan sebelum permintaan diteruskan ke Aggregator.

Untuk mempertahankan sesi setelah *access token* kedaluwarsa, Auth Service menerbitkan *refresh token* berbentuk *opaque token*. Nilai token dibuat menggunakan sumber acak kriptografis dan hanya *hash*-nya yang disimpan. *Endpoint* "POST /v1/token/refresh" tetap mewajibkan *client credential* yang sama dengan pemilik sesi. Rotasi dilakukan dalam *database transaction* menggunakan penguncian baris agar satu token tidak dapat dipakai oleh dua permintaan secara bersamaan. Token lama ditandai telah digunakan, token pengganti disimpan dalam *token family* yang sama, dan penggunaan ulang token lama menyebabkan seluruh *refresh session* dalam keluarga tersebut dicabut.



Gambar 6. Alur kedaluwarsa dan rotasi token

Pembatasan data dilakukan pada Client-Facing API sebagai *policy-enforcement point*. Data Ringkasan terdiri atas “hazard_id”, ”source”, “hazard_type”, “severity”, “area_name”, “occurred_at”, dan “ingested_at”. *Raw data* terdiri atas “source_ref_id”, “latitude”, “longitude”, dan “attributes”. Respons Media dibentuk menggunakan *allowlist field* Ringkasan. Apabila Media meminta field Mentah secara eksplisit melalui parameter “fields”, server menolak seluruh permintaan.



Gambar 7. Alur otorisasi *field* pada Client-Facing API

Client-Facing API mengakses Aggregator menggunakan “X-Aggregator-Key”, sehingga *client downstream* tidak menerima *key* tersebut. Aggregator tidak memublikasikan *port* ke *host* dan hanya dapat diakses melalui jaringan internal Docker. Pemisahan ini mencegah *client* melewati pemeriksaan JWT dan kebijakan *field* dengan mengakses Canonical Store secara langsung.

Seluruh API *key*, *client secret*, kata sandi *database*, dan *private signing key* dibaca dari *environment variable*. Berkas “.env” di-*ignore* oleh Git, dan “.env.example” hanya memuat daftar variabel dan nilai contoh. Setiap *container* hanya menerima secret yang diperlukan oleh *task*-nya. Nilai *header* autentikasi, *access token*, dan *refresh token* juga tidak dimasukkan ke log. Apabila kredensial tidak tersedia atau tidak valid, layanan menolak akses secara *fail-closed* dan tidak menggunakan kredensial bawaan.

Alasan Keputusan

Beberapa alternatif solusi telah dipertimbangkan sebagai berikut.

No.	Alternatif solusi	Kelebihan	Kekurangan
1	Menggunakan satu kredensial untuk BMKG, PVMBG, dan seluruh *client*.	Konfigurasi dan implementasi paling sederhana.	Kebocoran satu kredensial membuka seluruh domain dan meniadakan otonomi setiap *instance*.
2	Menggunakan *opaque access token* dengan introspeksi ke Auth Service pada setiap *request*.	Token dapat dicabut secara langsung dan status sesi selalu diperiksa.	Menambah satu *network call* dan menjadikan Auth Service dependensi untuk setiap *data read*.
3	Menggunakan JWT dengan algoritma HS256.	*Publish* dan verifikasi token relatif sederhana.	Auth Service dan Client-Facing API harus berbagi *secret* yang sama sehingga pihak yang dapat memverifikasi token juga dapat membuat token.
4	Menggunakan JWT RS256 berumur pendek dan *rotating opaque refresh token*.	Verifikasi dapat dilakukan secara lokal dan hanya Auth Service yang memiliki *private signing key*.	*Access token* yang masih berlaku tidak dapat dicabut secara instan dan pengelolaan rotasi token lebih kompleks.
5	Menggunakan Authorization Server OAuth/OIDC lengkap.	Menyediakan protokol standar untuk federasi identitas, *consent*, dan alur pengguna.	Menambah konsep *authorization code*, *redirect*, dan *resource owner* yang tidak diperlukan oleh tiga *machine client* tetap pada sistem ini.

Dari alternatif tersebut, dipilih alternatif nomor empat. JWT RS256 memungkinkan Client-Facing API memverifikasi token tanpa menghubungi Auth Service pada setiap permintaan, tetapi tidak memberinya kemampuan menerbitkan token. Risiko token yang tidak dapat dicabut secara langsung dibatasi dengan TTL 60 detik. *Refresh token* dipertahankan karena kriteria Problem 3 mewajibkan sesi dapat dilanjutkan tanpa login ulang manual setelah *access token* kedaluwarsa.

Implementasi tidak dinyatakan sebagai Authorization Server OAuth/OIDC lengkap. Sistem hanya menerapkan mekanisme yang diperlukan oleh permasalahan, yaitu autentikasi *machine client*, Bearer token, *scope*, JWT, serta rotasi *refresh token*. Pilihan tersebut menghindari komponen alur pengguna yang tidak ada pada domain masalah tanpa mengurangi pemisahan *trust domain*.

Batasan yang Tidak Ditangani

Dalam perancangan solusi permasalahan ini, terdapat beberapa hal yang secara sadar tidak ditangani, yaitu sebagai berikut.

- *Access token* yang belum kedaluwarsa tidak dapat dicabut secara instan. Risiko ini dibatasi menggunakan TTL pendek 60 detik. Pencabutan langsung memerlukan introspeksi atau distribusi *deny list* token yang akan menambah ketergantungan (*coupled*) antarlayanan.
- Implementasi hanya memiliki satu *signing key* aktif. *Endpoint* JWKS dan *claim* "kid" sudah tersedia, tetapi publikasi kunci lama dan baru secara bersamaan ketika rotasi belum diimplementasikan. Mekanisme tersebut belum diperlukan untuk membuktikan autentikasi dan pemisahan domain kepercayaan pada Problem 3.
- Docker Compose belum menyediakan terminasi TLS. Untuk lingkungan non-lokal, TLS perlu ditempatkan pada *reverse proxy* atau API *gateway* agar *client secret*, *access token*, dan *refresh token* tidak dikirim sebagai *plaintext* pada *network*.

Bukti Penyelesaian

Berikut adalah bukti penyelesaian Problem 3.

1. **Isolasi kredensial BMKG dan PVMBG**

   Pengujian diawali dengan *positive control* untuk menunjukkan bahwa setiap kredensial valid pada layanan pemiliknya. Setelah itu, kredensial yang sama di-*exchange* untuk membuktikan bahwa kedua *trust domain* benar-benar terisolasi.

   Berikut adalah hasil dari *positive control testing*:

Kredensial BMKG ke BMKG:

```
└─Δ curl -i \
  -H "X-BMKG-Key: ${BMKG_API_KEY}" \
  'http://localhost:8081/seismic-events?since=1970-01-01T00:00:00Z'
HTTP/1.1 200 OK
Content-Type: application/json
X-Correlation-Id: 8a30ee1efe9ccd3290a382e1d161e90f
Date: Sat, 10 Oct 2026 04:34:27 GMT
Transfer-Encoding: chunked

[{"event_id":"EQ-001","magnitude":5,"depth_km":8,"epicenter_lat":-9,"epicen
ter_lon":105,"region_name":"Jawa Barat","occurred_at":"2026-09-01T00:00:00Z
","potential_tsunami":true},{"event_id":"EQ-002","magnitude":5.15,"depth_km
":10.5,"epicenter_lat":-8.75,"epicenter_lon":105.4,"region_name":"Jawa Teng
```

Kredensial PVMBG ke PVMBG:

```
└─Δ curl -i \
  -H "Authorization: Bearer ${PVMBG_TOKEN}" \
  'http://localhost:8082/volcanic-reports?since=1970-01-01T00:00:00Z'
HTTP/1.1 200 OK
Content-Type: application/json
X-Correlation-Id: 0c689b515279d01a2bb22539045a6a2d
Date: Sat, 10 Oct 2026 04:35:33 GMT
Transfer-Encoding: chunked

[{"report_id":"VR-001","volcano_id":"MERAPI","alert_level":"Normal","erupti
on_count_24h":0,"ash_column_height_m":300,"reported_at":"2026-09-01T00:00:0
0Z"},{"report_id":"VR-002","volcano_id":"SEMERU","alert_level":"Waspada","e
```

Keluaran tersebut menunjukkan bahwa kredensial yang benar akan diterima, selanjutnya adalah pengujian *negative control*:

Kredensial BMKG ke PVMBG:

```
└─Δ curl -i \
  -H "Authorization: Bearer ${BMKG_API_KEY}" \
  'http://localhost:8082/volcanic-reports?since=1970-01-01T00:00:00Z'
HTTP/1.1 401 Unauthorized
Content-Type: text/plain; charset=utf-8
X-Content-Type-Options: nosniff
X-Correlation-Id: 930257b294cd477efa2231301d0bfc34
Date: Sat, 10 Oct 2026 04:36:59 GMT
Content-Length: 26

{"error":"invalid token"}
```

Kredensial PVMBG ke BMKG:

```
└─Δ curl -i \
  -H "X-BMKG-Key: ${PVMBG_TOKEN}" \
  'http://localhost:8081/seismic-events?since=1970-01-01T00:00:00Z'
HTTP/1.1 401 Unauthorized
Content-Type: text/plain; charset=utf-8
X-Content-Type-Options: nosniff
X-Correlation-Id: b86d4e2b0983c21ba7de69ed61a9c066
Date: Sat, 10 Oct 2026 04:38:30 GMT
Content-Length: 28

{"error":"invalid API key"}
```

Terlihat bahwa kredensial lintas-*instance* ditolak dengan me-*return* HTTP *code* 401 (*Unauthorized*).

2. Pembatasan field untuk Media

Token Media diminta terlebih dahulu melalui Auth Service. Nilai “access_token” dari respons digunakan untuk meminta respons bawaan dan *raw field* “latitude”. Sebagai *positive control* untuk *raw data*, token Tim Lapangan digunakan untuk membaca *event* yang sama.

Berikut adalah hasil dari *positive control testing*, dan sebelum itu, kita bisa menyimpan token dengan *command* berikut:

```
MEDIA_TOKEN_RESPONSE="$(
 curl -sS -X POST \
  -u "${MEDIA_CLIENT_ID}:${MEDIA_CLIENT_SECRET}" \
  http://localhost:8084/v1/token
)"

FIELD_TOKEN_RESPONSE="$(
 curl -sS -X POST \
  -u "${FIELD_CLIENT_ID}:${FIELD_CLIENT_SECRET}" \
  http://localhost:8084/v1/token
)"

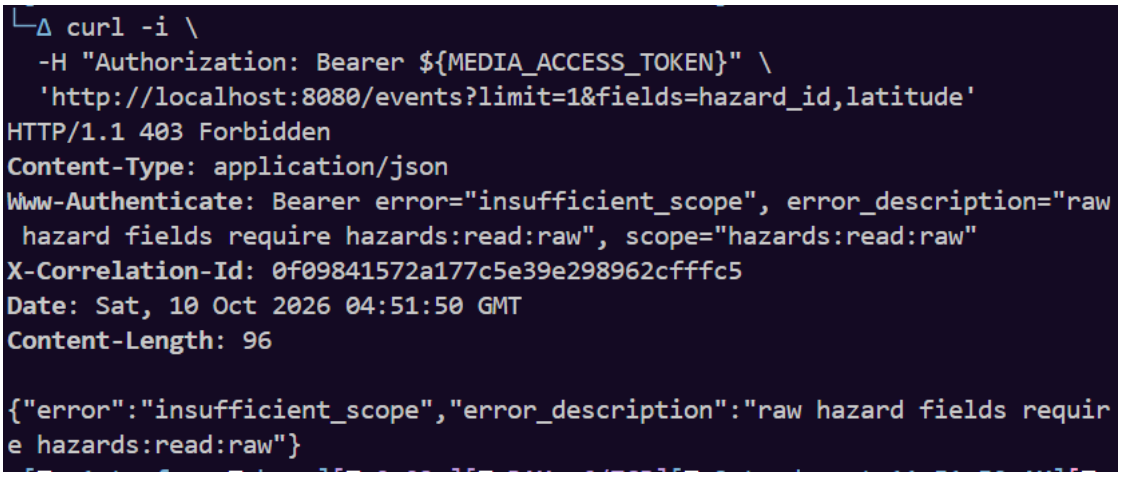
MEDIA_ACCESS_TOKEN="$(
 jq -er '.access_token' <<<"${MEDIA_TOKEN_RESPONSE}"
)"
```

```
FIELD_ACCESS_TOKEN="$(
  jq -er '.access_token' <<<"${FIELD_TOKEN_RESPONSE}"
)"
```

Token Media valid dan dapat membaca *field* Ringkasan:



Media meminta field Mentah:



Tim Lapangan membaca field Mentah:



“source_ref_id”, “latitude”, “longitude”, dan “attributes” tidak terdapat pada respons Media, tetapi terdapat pada respons Tim Lapangan. Dengan demikian, field Mentah memang tersedia pada sistem dan penghilangannya dari respons Media terjadi karena kebijakan server, bukan karena *field* tersebut tidak ada pada data sumber.

3. Kedaluwarsa dan *refresh token* Tim Lapangan

Token Tim Lapangan terlebih dahulu digunakan ketika masih berlaku. Nilai “expires_in”, “access_token”, dan “refresh_token” diperoleh dari respons Auth Service. Setelah menunggu 60 detik tanpa mengubah konfigurasi TTL, *refresh token* digunakan untuk memperoleh pasangan token baru. Proses ini tidak memerlukan login pengguna secara manual, tetapi tetap mengautentikasi *machine client* menggunakan *client credential* yang sama.

Untuk mengambil token Tim Lapangan, digunakan *command* berikut:

```
FIELD_TOKEN_RESPONSE="$(
  curl -sS -X POST \
    -u "${FIELD_CLIENT_ID}:${FIELD_CLIENT_SECRET}" \
    http://localhost:8084/v1/token
)"

OLD_ACCESS_TOKEN="$(jq -er '.access_token'
<<<"${FIELD_TOKEN_RESPONSE}")"
OLD_REFRESH_TOKEN="$(jq -er '.refresh_token'
```

```
<<<"${FIELD_TOKEN_RESPONSE}")"
```

Berikut adalah pengujian token sebelum kedaluwarsa:

```
└─Δ curl -i \
  -H "Authorization: Bearer ${OLD_ACCESS_TOKEN}" \
  'http://localhost:8080/events?limit=1'
HTTP/1.1 200 OK
Content-Type: application/json
X-Correlation-Id: f7fe2800bec4c6498793b9a32a2eb018
Date: Sat, 10 Oct 2026 05:24:28 GMT
Content-Length: 498

{"events":[{"area_name":"Demo Region","attributes":{"depth_km":15,"magnitude"
:6.95,"potential_tsunami":false},"hazard_id":"HE-636","hazard_type":"SEISMIC"
,"ingested_at":"2026-10-10T05:24:23.250576Z","latitude":-6.2,"longitude":106.
8,"occurred_at":"2026-10-10T05:24:21.794347Z","severity":"SIAGA","source":"BM
KG","source_ref_id":"EQ-317"}],"sources":{"BMKG":{"state":"UP","last_success_
at":"2026-10-10T05:24:28.273189219Z"},"PVMBG":{"state":"UP","last_success_at"
:"2026-10-10T05:24:25.13605225Z"}}}
```

Dan berikut adalah pengujian setelah kedaluwarsa:

```
┌[ w1ntr from  kaze][ 0.101s][ RAM: 6/7GB][ Saturday at 12:24:28 PM][ 
main ≡]
└[/mnt/c/Users/Ghana/Documents/aat/IF4031-M1-AAT]
└─Δ sleep 60
┌[ w1ntr from  kaze][ 1:0.024s][ RAM: 6/7GB][ Saturday at 12:25:31 PM][
 main ≡]
└[/mnt/c/Users/Ghana/Documents/aat/IF4031-M1-AAT]
└─Δ curl -i \
  -H "Authorization: Bearer ${OLD_ACCESS_TOKEN}" \
  'http://localhost:8080/events?limit=1'
HTTP/1.1 401 Unauthorized
Content-Type: application/json
Www-Authenticate: Bearer error="invalid_token", error_description="access tok
en is invalid or expired"
X-Correlation-Id: 4c6ea7f961eb6401637f83e78c498922
Date: Sat, 10 Oct 2026 05:25:34 GMT
Content-Length: 83

{"error":"invalid_token","error_description":"access token is invalid or expi
red"}
```

Terlihat bahwa token menjadi invalid setelah 60 detik. Selanjutnya lakukan *refresh token* (*flow* belum dilakukan secara otomatis untuk kebutuhan *Proof of Concept* dan pengujian) tanpa melakukan login ulang:

```
REFRESH_RESPONSE="$(
 jq -n \
   --arg refresh_token "${OLD_REFRESH_TOKEN}" \
   '{refresh_token: $refresh_token}' \
 | curl -fsS -X POST \
    -u "${FIELD_CLIENT_ID}:${FIELD_CLIENT_SECRET}" \
    -H 'Content-Type: application/json' \
    --data-binary @- \
    http://localhost:8084/v1/token/refresh
)"

NEW_ACCESS_TOKEN="$(jq -er '.access_token' <<<"${REFRESH_RESPONSE}")"
NEW_REFRESH_TOKEN="$(jq -er '.refresh_token'
<<<"${REFRESH_RESPONSE}")"
```

Dan berikut adalah hasil dari *request* dengan *refreshed token*-nya:

```
└─Δ REFRESH_RESPONSE="$(
  jq -n \
    --arg refresh_token "${OLD_REFRESH_TOKEN}" \
    '{refresh_token: $refresh_token}' \
  | curl -fsS -X POST \
      -u "${FIELD_CLIENT_ID}:${FIELD_CLIENT_SECRET}" \
      -H 'Content-Type: application/json' \
      --data-binary @- \
      http://localhost:8084/v1/token/refresh
)"

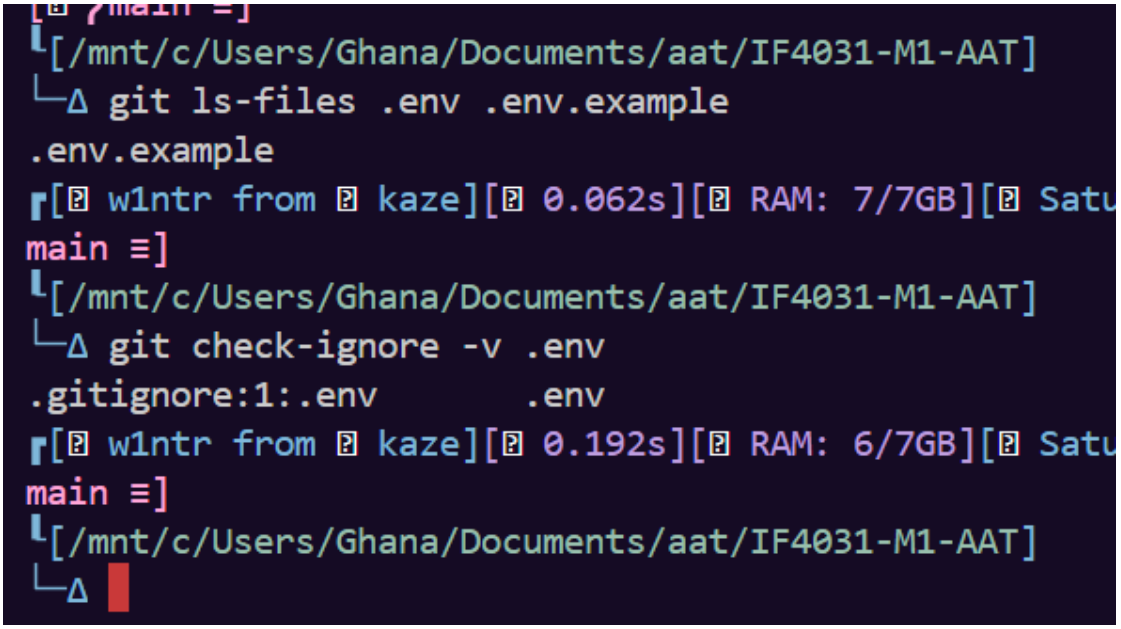
NEW_ACCESS_TOKEN="$(jq -er '.access_token' <<<"${REFRESH_RESPONSE}")"
NEW_REFRESH_TOKEN="$(jq -er '.refresh_token' <<<"${REFRESH_RESPONSE}")"
┌[ w1ntr from  kaze][ 0.361s][ RAM: 6/7GB][ Saturday at 12:35:19 PM][ 
main ≡]
└[/mnt/c/Users/Ghana/Documents/aat/IF4031-M1-AAT]
└─Δ curl -i \
  -H "Authorization: Bearer ${NEW_ACCESS_TOKEN}" \
  'http://localhost:8080/events?limit=1'
HTTP/1.1 200 OK
Content-Type: application/json
X-Correlation-Id: 0b10f99e36603b3bb71e1740b6febb7c
Date: Sat, 10 Oct 2026 05:35:24 GMT
Content-Length: 499

{"events":[{"area_name":"Demo Region","attributes":{"depth_km":17,"magnitude"
:7.25,"potential_tsunami":false},"hazard_id":"HE-768","hazard_type":"SEISMIC"
,"ingested_at":"2026-10-10T05:35:23.205239Z","latitude":-6.2,"longitude":106.
8,"occurred_at":"2026-10-10T05:35:21.794347Z","severity":"SIAGA","source":"BM
KG","source_ref_id":"EQ-383"}],"sources":{"BMKG":{"state":"UP","last_success_
at":"2026-10-10T05:35:23.261252905Z"},"PVMBG":{"state":"UP","last_success_at"
:"2026-10-10T05:35:24.324269315Z"}}}
```

Terlihat bahwa *request* berhasil diterima dengan HTTP *code* 200.

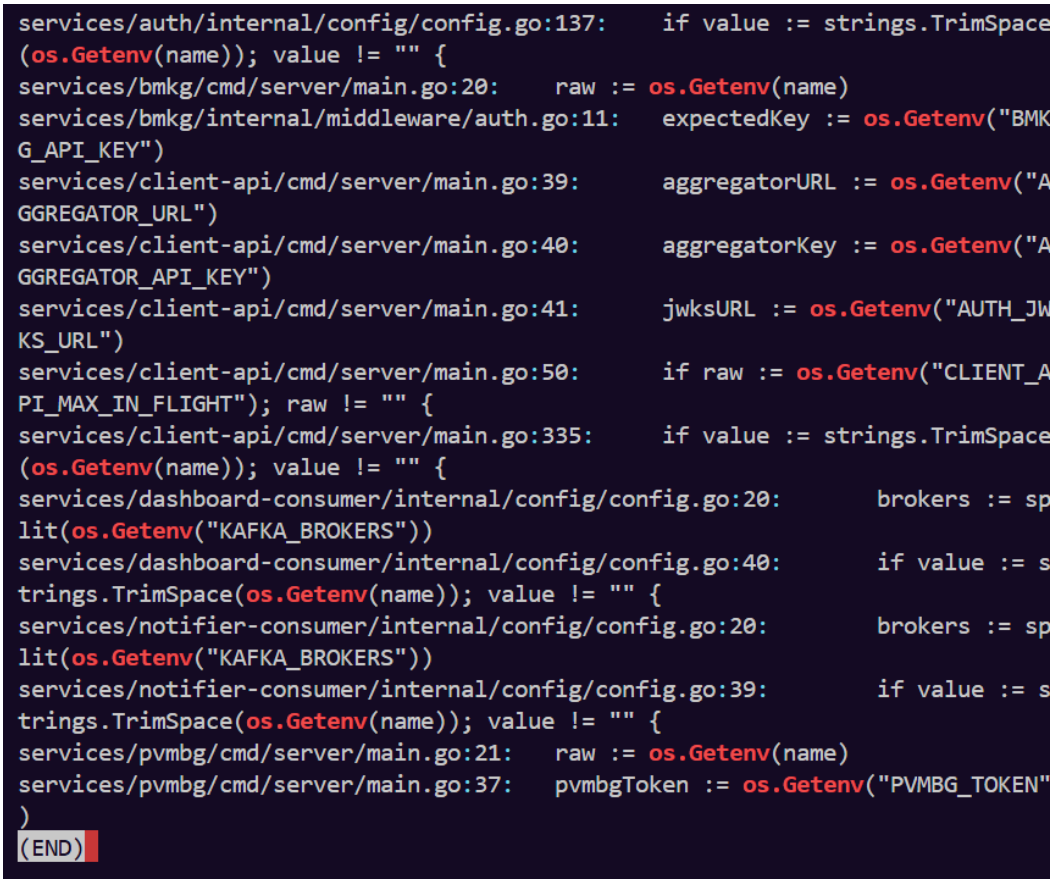
4. Pengelolaan *secret* dan konfigurasi

Berikut adalah hasil dari pengecekan bahwa tidak ada kredensial yang ter-*hardcode* di kode maupun ter-*commit*, dan .env.example tersedia sesuai Ketentuan Implementasi.



```
└─Δ git grep -n -E 'os\.(Getenv|LookupEnv)' -- services
services/aggregator/internal/config/config.go:39:       databaseURL := string
s.TrimSpace(os.Getenv("DATABASE_URL"))
services/aggregator/internal/config/config.go:45:       if raw := os.Getenv("
AGGREGATOR_START_SINCE"); raw != "" {
services/aggregator/internal/config/config.go:55:               KafkaBrokers:
    splitCommaSeparated(os.Getenv("KAFKA_BROKERS"), "localhost:9092"),
services/aggregator/internal/config/config.go:58:               BMKGAPIKey:
    os.Getenv("BMKG_API_KEY"),
services/aggregator/internal/config/config.go:60:               PVMBGToken:
    os.Getenv("PVMBG_TOKEN"),
services/aggregator/internal/config/config.go:69:       raw := os.Getenv(name
)
services/aggregator/internal/config/config.go:82:       raw := os.Getenv(name
)
services/aggregator/internal/config/config.go:94:       if value := os.Getenv
(name); value != "" {
services/aggregator/internal/handler/auth.go:11:        expectedKey := os.Get
env("AGGREGATOR_API_KEY")
services/auth/internal/config/config.go:52:             {ID: os.Getenv("MEDIA
_CLIENT_ID"), Secret: os.Getenv("MEDIA_CLIENT_SECRET"), Scopes: []string{"haz
ards:read:summary"}},
services/auth/internal/config/config.go:53:             {ID: os.Getenv("FIELD
_CLIENT_ID"), Secret: os.Getenv("FIELD_CLIENT_SECRET"), Scopes: []string{"haz
ards:read:summary", "hazards:read:raw"}},
services/auth/internal/config/config.go:54:             {ID: os.Getenv("OPS_C
LIENT_ID"), Secret: os.Getenv("OPS_CLIENT_SECRET"), Scopes: []string{"hazards
:read:summary", "hazards:read:raw"}},
services/auth/internal/config/config.go:62:             DatabaseURL:    os.Ge
tenv("AUTH_DATABASE_URL"),
services/auth/internal/config/config.go:79:     raw := strings.TrimSpace(os.G
etenv("AUTH_JWT_PRIVATE_KEY_B64"))
services/auth/internal/config/config.go:113:    raw := os.Getenv(name)
services/auth/internal/config/config.go:125:    raw := os.Getenv(name)
services/auth/internal/config/config.go:137:    if value := strings.TrimSpace
```

```
└─Δ git grep -n -I -P \
    '(?i)(password|secret|token|api.?key)\s*[:=]\s*\x22[^\x22]+\x22' \
    -- '*.go' ':(exclude)*_test.go'
┌[ w1ntr from  kaze][ 0.23s][ RAM: 6/7GB][ Saturday at 12:45:22 PM][ ⎇r
ain ≡][ Error, check your command]
└[/mnt/c/Users/Ghana/Documents/aat/IF4031-M1-AAT]
└─Δ git grep -n -I -P \
    '(-----BEGIN (RSA )?PRIVATE KEY-----|postgres(ql)?://[^/:\s]+:[^@\s]+@)'
 \
    -- . ':(exclude)*.md' ':(exclude).env.example' ':(exclude)*_test.go'
┌[ w1ntr from  kaze][ 0.261s][ RAM: 6/7GB][ Saturday at 12:45:27 PM][
```



Tidak ada kredensial yang bocor atau ter-*hardcode* di kode maupun yang ter-*commit*,

Jawaban terhadap Butir Permasalahan

Jawaban atas butir kelima Problem 3 adalah sebagai berikut.

- Mekanisme yang dipilih terdiri atas pemisahan kredensial per *trust domain*, Auth Service khusus, JWT RS256 berumur pendek, JWKS untuk distribusi *public key*, *rotating opaque refresh token*, *scope* yang ditentukan oleh server, serta *allow list field* pada Client-Facing API.
- Mekanisme tersebut tidak membuka kredensial universal karena BMKG, PVMBG, Aggregator, dan setiap *client downstream* memiliki kredensial serta validator yang berbeda. Kebocoran API *key* BMKG tidak memberikan akses ke PVMBG atau Client-Facing API. Kebocoran *client secret* Media juga tidak memberikan akses ke

field Mentah karena *scope* Media ditentukan oleh Auth Service dan pembatasan *field* dilakukan oleh server.

- Penggunaan RS256 membuat *private signing key* hanya tersedia pada Auth Service. Client-Facing API hanya menerima *public key*, sehingga komponen tersebut dapat memverifikasi tetapi tidak dapat menerbitkan token. JWT diverifikasi secara lokal agar Auth Service tidak menjadi dependensi pada setiap *request* dan dibatasi dengan TTL 60 detik.
- *Refresh token* dibuat secara acak, disimpan dalam bentuk *hash*, dirotasi secara transaksional, dan dilengkapi deteksi penggunaan ulang. Pembatasan *field* menggunakan *allow list* sehingga penambahan *field* baru tidak otomatis membocorkan data kepada Media. Dengan kombinasi tersebut, kompromi pada satu domain tidak memberikan akses otomatis ke domain lain.

Problem 4: Arsitektur Microservice & *Flexible Storage*

Masalah keempat menyatakan bahwa layanan yang diberikan oleh sistem belum independen, menyebabkan perlunya *redeploy* komponen-komponen yang tidak berhubungan ketika mengubah salah satu *service* saja. Terdapat juga masalah lain mengenai penyimpanan HazardEvent, tabel basis data memiliki struktur atribut tetap yang tidak mendukung penambahan atribut dari instansi sumber.

Rancangan Solusi

Solusi yang dipilih untuk menyelesaikan permasalahan terkait independensi layanan adalah dengan mendesain sistem menggunakan arsitektur *microservices* yang memisahkan setiap komponen pada sistem melalui *container* yang berbeda menggunakan Docker. Seperti yang dijelaskan pada bagian arsitektur sistem, sistem terdiri atas 10 komponen yang memiliki fungsinya masing-masing. Sepuluh komponen tersebut dibuat dalam *container* terpisah sehingga masing-masing komponen dapat di-*deploy* atau dimatikan secara independen. Dengan begitu, perubahan pada satu atau lebih komponen yang mengurus suatu layanan tidak mengharuskan *redeploy* total, melainkan hanya perlu *redeploy* komponen yang terkait saja. Solusi ini mengharuskan tiap komponen untuk berkomunikasi melalui jaringan, seperti menggunakan HTTP/REST API atau message broker.

Untuk menyelesaikan permasalahan terkait penyimpanan, komponen database pada sistem dibuat dengan menyimpan tabel relasional yang mengakomodasi penyimpanan nilai JSON (menggunakan JSONB) pada salah satu atributnya yang bernama attributes. Digunakanlah PostgreSQL yang menyediakan jenis penyimpanan tersebut. Dengan begitu, walaupun tabel basis data memiliki atribut tetap, terdapat atribut tambahan yang bisa menyimpan banyak jenis atribut sehingga ketika terdapat *field* baru yang ditambahkan dari instansi sumber, *field* tersebut tetap dapat terpetakan ke struktur basis data dengan memasukkannya ke dalam atribut attributes.

Alasan Keputusan

Beberapa alternatif solusi untuk masalah tersebut telah dipertimbangkan. Berikut adalah alternatif solusi untuk masalah terkait independensi layanan.

No.	Alternatif solusi	Kelebihan	Kekurangan
1	Desain sistem menggunakan arsitektur monolitik	Tidak ada *overhead* pengimplementasian komunikasi tiap layanan melalui jaringan (seperti *encoding*/*decoding* data, autentikasi, pengendalian kegagalan komunikasi, dan lain sebagainya), *bug* pada sistem mudah dilacak dan ditangani.	Layanan-layanan pada sistem menjadi *tightly coupled* (sangat bergantung satu sama lain) sehingga menyulitkan pengerjaan secara paralel, perubahan satu layanan memerlukan *redeploy* total, layanan yang rusak dapat menyebabkan keseluruhan sistem rusak.
2	Desain sistem menggunakan arsitektur *microservices*	Setiap layanan bisa dikerjakan secara independen (sangat mendukung pengerjaan paralel antarlayanan), perubahan satu layanan tidak mengharuskan *redeploy* total (hanya layanan yang terkait saja yang perlu *redeploy*), satu layanan rusak tidak mengakibatkan seluruh sistem rusak.	Setiap layanan harus mengimplementasikan komunikasi melalui jaringan (*encoding*/*decoding* data, autentikasi, pengendalian kegagalan komunikasi, dan lain sebagainya), *bug* menjadi sulit dilacak karena melibatkan berbagai layanan yang terpisah.

Berikut adalah alternatif solusi untuk masalah terkait penyimpanan.

No.	Alternatif solusi	Kelebihan	Kekurangan
1	Komponen database menyimpan tabel relasional dengan atribut attributes bertipe JSONB.	Tidak perlu ada perubahan skema tabel sama sekali ketika atribut baru muncul, skema tabel terdefinisi dengan jelas dan tidak berubah-ubah.	Tipe data atribut baru tidak jelas terdefinisi, atribut baru tidak dapat dikenakan *constraint*, *query* terhadap atribut baru menjadi kompleks.
2	Menggunakan mekanisme schema evolution pada tabel relasional yang disimpan database.	Struktur data tiap atribut eksplisit dan jelas, *query* terhadap atribut baru lebih mudah, dapat menambah *constraint* terhadap atribut baru.	Perlu ada perubahan skema tabel yang menyebabkan migrasi, skema tabel menjadi sangat rumit ketika terjadi banyak kemunculan atribut baru.
3	Menyimpan keseluruhan record HazardEvent dalam bentuk JSON.	Struktur *record* sangat fleksibel (memungkinkan penambahan, pengurangan, serta perubahan atribut), tidak perlu mendefinisikan seluruh atribut yang ada.	Struktur data setiap *record* pada basis data tidak jelas karena tidak ada definisi spesifik terkait atribut yang harus ada di tiap *record* sehingga harus dilakukan penanganan struktur data yang bisa saja berbeda-beda.

Alternatif solusi 2 pada permasalahan independensi layanan dipilih karena menjawab permasalahan terkait *redeploy* layanan tanpa mengganggu layanan lain. Alternatif solusi 1 pada permasalahan penyimpanan dipilih karena skema tabel yang terdefinisi dengan jelas dan tetap sehingga tidak perlu ada penyesuaian perubahan skema oleh layanan lain.

Batasan yang Tidak Ditangani

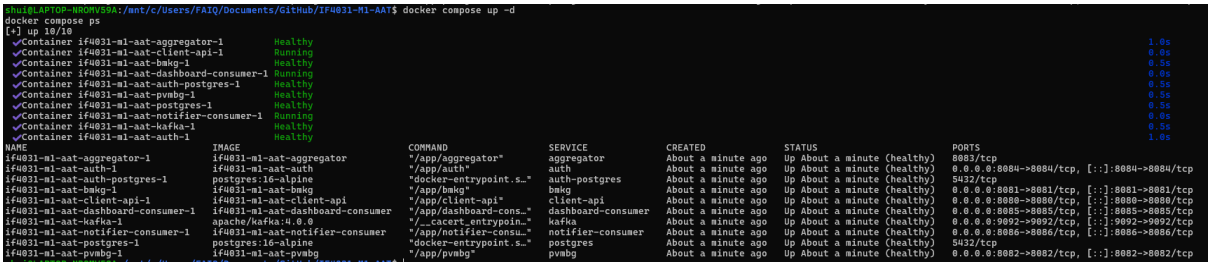
Dalam perancangan solusi permasalahan ini, terdapat beberapa hal yang secara sadar tidak ditangani, yaitu sebagai berikut.

- Penghilangan keperluan kontrak antarkomponen.
- Pengubahan nama *field* yang sudah ada di kontrak awal (contoh, mengubah field "region_name" menjadi "region" atau "occured_at" menjadi "time_occured").
- Penghapusan *field* yang sudah ada di kontrak awal.
- Pengubahan tipe data *field* (contoh, field depth_km yang awalnya bertipe float diubah menjadi tipe integer).

Bukti Penyelesaian

1. **Tiap komponen berjalan pada container terpisah. Salah satu komponen dapat dihentikan, di-*rebuild*, dan dijalankan ulang tanpa mengganggu komponen lain.**
    - Nyalakan dan perlihatkan status setiap komponen.

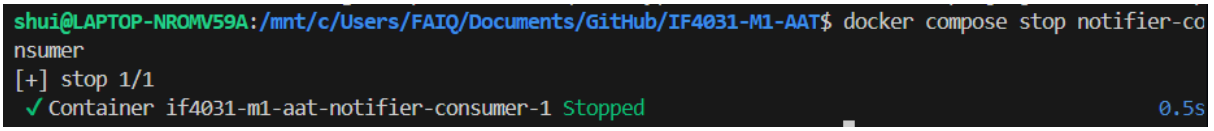
    Command: docker compose up -d
    docker compose ps

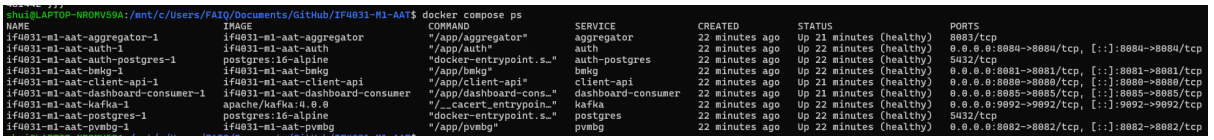


    - Matikan salah satu komponen, dalam hal ini notifier-consumer. Amati bahwa status komponen notifier-consumer setelah menjalankan *command* tidak ditemukan.

Command: docker compose stop notifier-consumer

docker compose ps





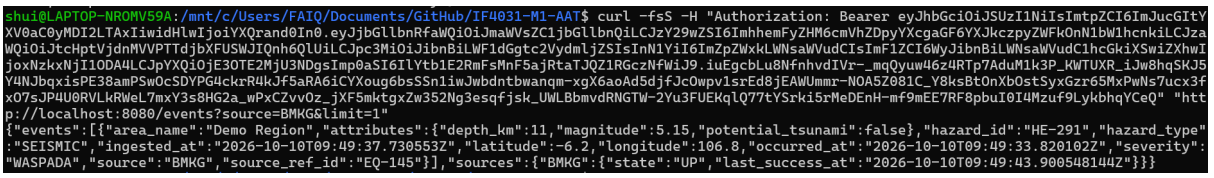
- Coba lakukan pengambilan token dari komponen authenticator sebagai tim lapangan, gunakan FIELD_CLIENT_ID dan FIELD_CLIENT_SECRET sesuai pada .env *file*.

Command: curl -fsS -X POST -u "<FIELD_CLIENT_ID>:<FIELD_CLIENT_SECRET>" http://localhost:8084/v1/token



- Coba lakukan pengambilan data melalui client-facing API menggunakan token yang didapat.

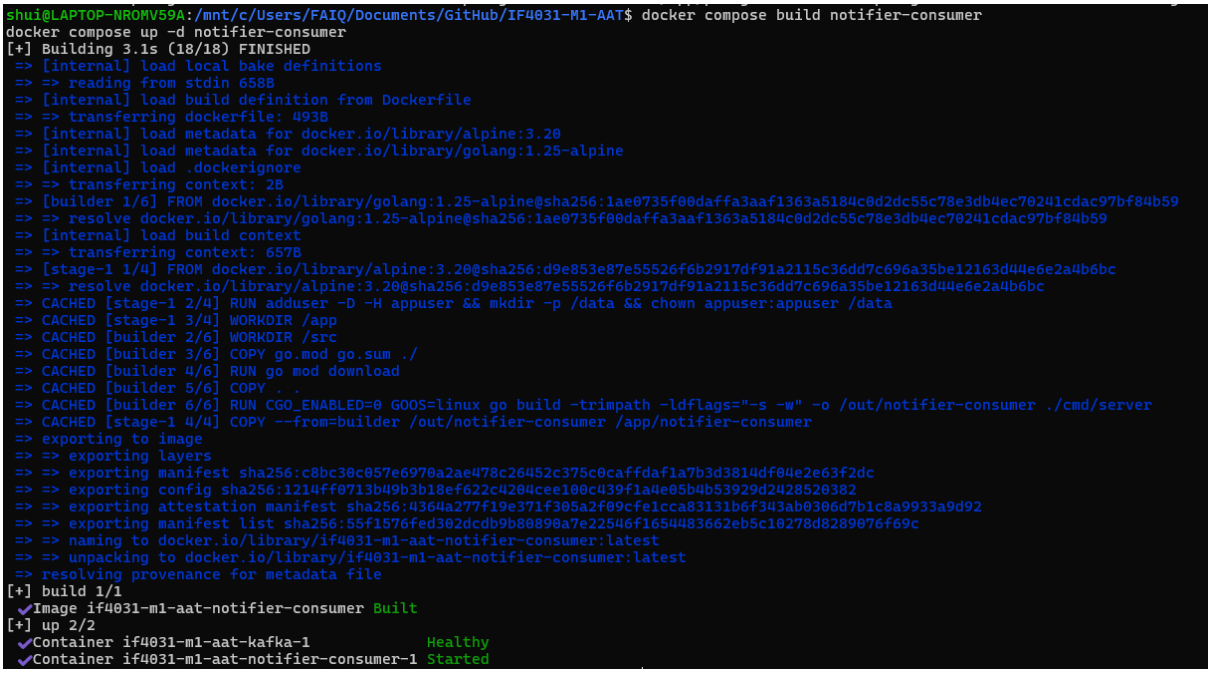
Command: curl -fsS -H "Authorization: Bearer <FIELD_ACCESS_TOKEN>" "http://localhost:8080/events?source=BMKG&limit=1"



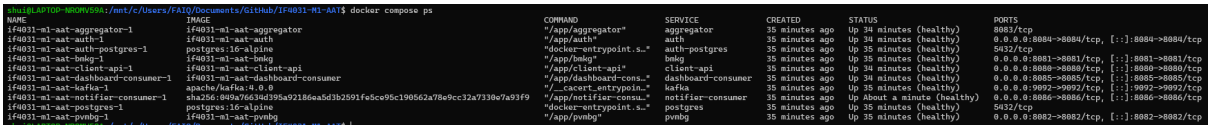
- Lakukan *redeploy* terhadap komponen yang dimatikan, dalam hal ini notifier-consumer

Command: docker compose build notifier-consumer

docker compose up -d notifier-consumer

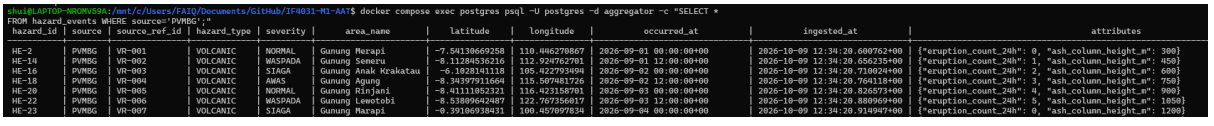


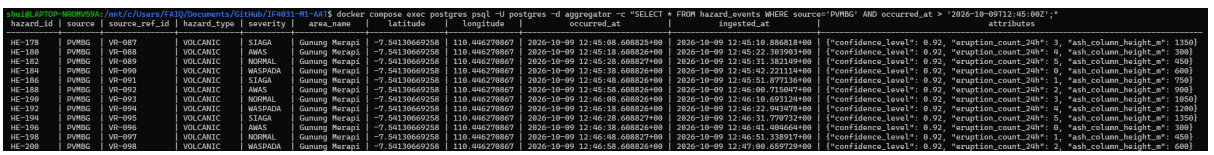
- Cek kembali status setiap komponen, perhatikan bahwa notifier-consumer dapat ditemukan di hasil *command*.



2. Penyimpanan HazardEvent mengakomodasi atribut dinamis tanpa perlu proses migrasi.

Bukti lengkap dapat dilihat pada keseluruhan bukti penyelesaian pada *problem* 1. Berikut adalah penyimpanan *record* sebelum dan setelah perubahan skema pada PVMBG.





3. ***Canonical store* pada komponen database hanya bisa diakses langsung oleh aggregator, komponen lain memperoleh data melalui API aggregator.**
    - Cek bahwa hanya komponen aggregator dan database terhubung melalui *network* khusus. Perhatikan bahwa hasil *command* memberikan daftar *container* yang terhubung yang terdiri atas aggregator dan database (postgres) saja.

Command: docker network inspect if4031-m1-aat_database

```
shui@LAPTOP-NROMV59A:/mnt/c/Users/FAIQ/Documents/GitHub/IF4031-M1-AAT$ docker network inspect if4031-m1-aat_database
[
    {
        "Name": "if4031-m1-aat_database",
        "Id": "e9b619da491505b6d42f363030b12c0652e507de80816289b794c8f5718dc2b8",
        "Created": "2026-10-10T16:27:58.479850995+07:00",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "172.19.0.0/16",
                    "Gateway": "172.19.0.1"
                }
            ]
        },
        "Internal": true,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {},
        "Labels": {
            "com.docker.compose.config-hash": "20cca005cf2e3188c5f662af1d1c267af126ba7440496a06c5619199fba8f801",
            "com.docker.compose.network": "database",
            "com.docker.compose.project": "if4031-m1-aat",
            "com.docker.compose.version": "5.5.1"
        },
        "Containers": {
            "2b244158072b110c0bb7caf242488d8f08c1bc217745f1dc1570f09ff572a9a5": {
                "Name": "if4031-m1-aat-aggregator-1",
                "EndpointID": "df9bd4911637 4a83861fdf72f5040cfaaba03e313203463864228978ac227212",
                "MacAddress": "ae:ec:1a:8b:41:f4",
                "IPv4Address": "172.19.0.3/16",
                "IPv6Address": ""
            },
            "bd78e9d9836603c6975ff07f690b5346e4637befa6e83c42d816f62b88efdbb5": {
                "Name": "if4031-m1-aat-postgres-1",
                "EndpointID": "dba20b1529ab796e3f2b72b8c7f027076f35b5a5afd43e90c44a8b81dea5aa5d",
                "MacAddress": "ee:a4:49:5b:75:64",
                "IPv4Address": "172.19.0.2/16",
                "IPv6Address": ""
            }
        },
        "Status": {
            "IPAM": {
                "Subnets": {
                    "172.19.0.0/16": {
                        "IPsInUse": 5,
                        "DynamicIPsAvailable": 65531
                    }
                }
            }
        }
    }
]
```

- Cek bahwa komponen database (postgres) hanya bisa ditemukan *IP address*-nya melalui komponen aggregator saja. Perhatikan bahwa *IP*

address ditemukan ketika *command* menggunakan komponen aggregator untuk mendapatkan *IP address* database.

Command: docker compose exec auth getent hosts postgres

docker compose exec auth-postgres getent hosts postgres

docker compose exec bmkg getent hosts postgres

docker compose exec client-api getent hosts postgres

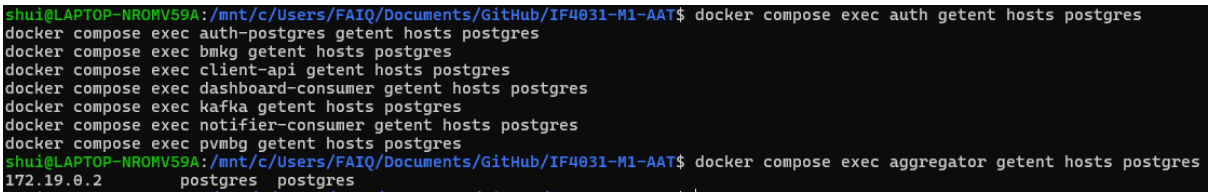
docker compose exec dashboard-consumer getent hosts postgres

docker compose exec kafka getent hosts postgres

docker compose exec notifier-consumer getent hosts postgres

docker compose exec pvmbg getent hosts postgres

docker compose exec aggregator getent hosts postgres



Jawaban terhadap Butir Permasalahan

- (Butir 4) Pemecahan sistem menjadi 10 komponen tersebut dipilih berdasarkan tanggung jawab yang jelas terdefinisi pada spesifikasi, sehingga tidak perlu memikirkan pembagian tanggung jawab baru yang berbeda dalam mendesain sistem yang ingin dibuat. Semua komponen yang dibuat di sistem terdapat pada spesifikasi kecuali komponen authenticator database yang dibutuhkan untuk menyimpan token yang dibangkitkan komponen authenticator.

  Alternatif penyimpanan dapat dilihat secara lengkap pada bagian alasan keputusan pada penjelasan penyelesaian problem ini, yaitu pada tabel kedua nomor 2 dan 3. Berikut adalah konsekuensi memilih penyimpanan alternatif.

  - Menggunakan mekanisme schema evolution pada tabel relasional yang disimpan database: basis data harus melakukan migrasi setiap kali atribut pada tabel bertambah, skema tabel bisa jadi rumit ketika banyak atribut baru yang ditambahkan oleh instansi sumber.

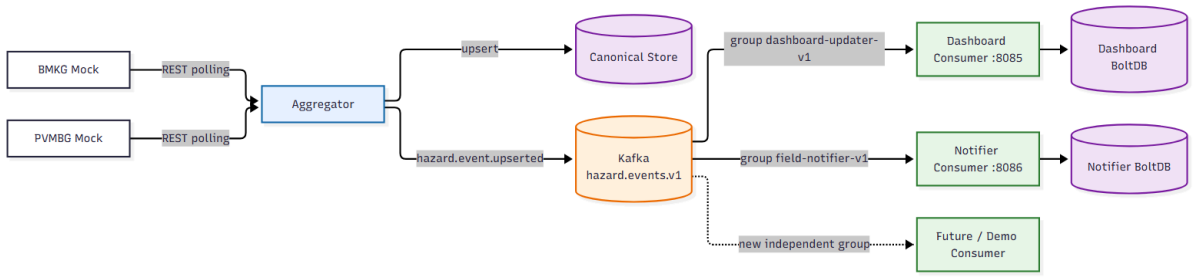
- Menyimpan keseluruhan record HazardEvent dalam bentuk JSON: struktur data setiap *record* pada basis data tidak jelas sehingga harus dilakukan penanganan struktur data yang bisa saja berbeda-beda.

Problem 5: *Fan-out Event Distribution/Pub-sub*

Problem 5 membahas keterkaitan langsung antara Aggregator sebagai penghasil "HazardEvent" dan pihak-pihak yang membutuhkan *event* tersebut. Jika Aggregator memanggil Dashboard, Notifier, dan *consumer* lain secara sinkron, keterlambatan satu *consumer* dapat menahan *consumer* lain. Penambahan *consumer* juga mengharuskan perubahan pada Aggregator. Sehingga, dibutuhkan solusi untuk memisahkan *producer* dari *consumer*, mempertahankan *event* ketika salah satu *consumer* berhenti, dan memungkinkan *subscriber* baru ditambahkan tanpa mengubah *producer*.

Rancangan Solusi

Solusi menggunakan Apache Kafka sebagai *message broker* dengan pola *publish-subscribe*. Aggregator hanya mengetahui alamat *broker* dan *topic* "hazard.events.v1". Setelah data BMKG atau PVMBG dipetakan menjadi "HazardEvent" dan berhasil disimpan ke Canonical Store, Aggregator membungkusnya dalam *event* "hazard.event.upserted" lalu memublikasikannya ke Kafka. Aggregator tidak menyimpan alamat Dashboard Consumer maupun Notifier Consumer dan tidak memanggil API keduanya.



Gambar 8. Arsitektur distribusi *event*

Setiap pesan Kafka menggunakan "hazard_id" sebagai *key*. Kafka menggunakan *key* tersebut untuk menempatkan pembaruan *hazard* yang sama pada partisi yang sama sehingga urutannya terjaga per *hazard*. Konfigurasi *publisher* menggunakan pengiriman sinkron dan "RequiredAcks=all". *Event* yang dikirim menggunakan *envelope* berikut.

Field	Fungsi
“event_type”	Menyatakan jenis *event*, yaitu “hazard.event.upserted”.
“schema_version”	Menyatakan versi kontrak *event*.
“correlation_id”	Menghubungkan proses *polling*, penyimpanan, publikasi, dan pemrosesan *consumer*.
“published_at”	Menyatakan waktu publikasi ke *broker*.
“data”	Memuat “HazardEvent” kanonik.

Selain itu, “event_type”, “schema_version”, dan “correlation_id” juga disalin ke *header* Kafka. Dengan demikian, *consumer* dapat melakukan *routing* dan observasi tanpa harus membaca seluruh *payload* terlebih dahulu.

Dua *consumer* independen membaca *topic* yang sama menggunakan *consumer group* berbeda.

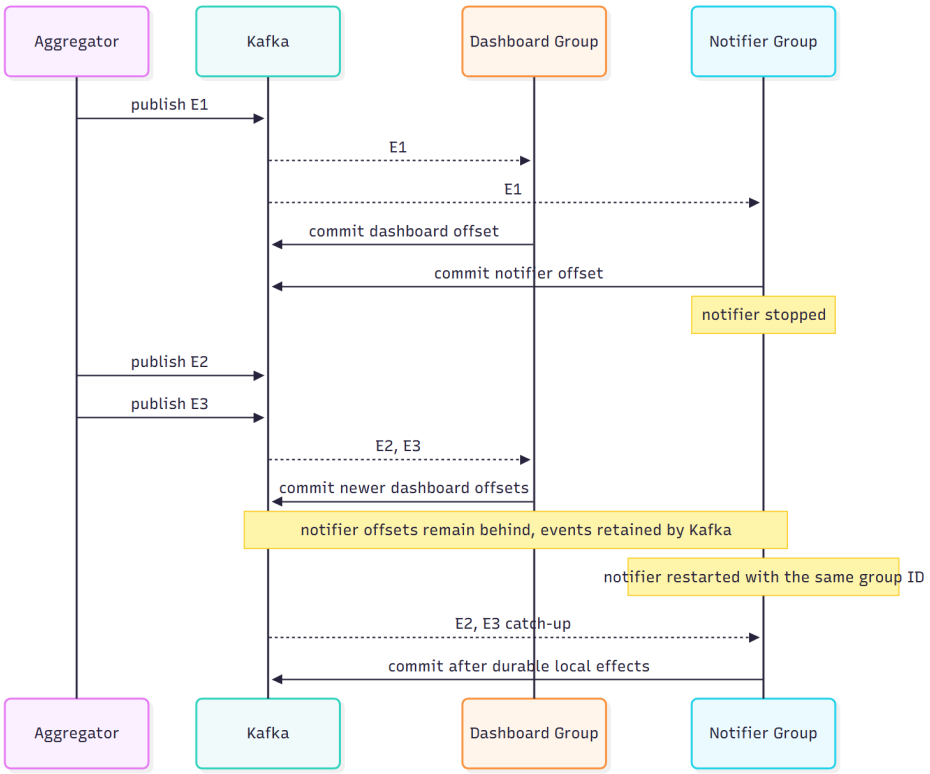
Consumer	*Consumer Group*	Hasil Pemrosesan yang Disimpan
Dashboard Consumer	“dashboard-updater-v1”	Menyimpan keadaan terbaru setiap “HazardEvent” pada *embedded database* bbolt milik Dashboard Consumer.
Notifier Consumer	“field-notifier-v1”	Membentuk dan menyimpan notifikasi lapangan pada *embedded database* bbolt milik Notifier Consumer.

Perbedaan *group* menyebabkan setiap *group* memperoleh *logical copy* dari seluruh *event*. Jika beberapa *instance* ditempatkan dalam *group* yang sama, Kafka membagi partisi antar-*instance* sehingga setiap pesan diproses oleh satu anggota *group* tersebut. Oleh karena itu, *group* berbeda digunakan untuk fungsi bisnis berbeda, sedangkan penambahan *replica* dalam *group* yang sama digunakan untuk membagi beban fungsi yang sama.

Kedua *consumer* menggunakan “FetchMessage” dan melakukan *commit offset* secara manual. Offset baru di-*commit* setelah transaksi bbolt berhasil menyimpan hasil pemrosesan *event* dan *message* ID. Jika proses berhenti setelah transaksi bbolt selesai tetapi sebelum *offset* berhasil di-*commit*, Kafka dapat mengirim pesan yang sama kembali. Kondisi ini membentuk semantik *at-least-once* untuk *event* valid.

Consumer dibuat idempoten untuk menghadapi pengiriman ulang tersebut. Keduanya mencatat identitas pesan berupa kombinasi “topic:partition:offset” dalam transaksi yang sama dengan hasil pemrosesan *event*. *Dashboard* menyimpan *event* menggunakan “hazard_id” sebagai *key* sehingga pembaruan *hazard* yang sama mengganti keadaan lama. *Notifier* menggunakan *key* domain “event_type:hazard_id:severity” sehingga pengiriman ulang tidak menghasilkan notifikasi domain yang sama, tetapi perubahan *severity* tetap dapat menghasilkan notifikasi baru.

Ketika Notifier Consumer dihentikan, Aggregator tetap menulis ke Kafka dan Dashboard Consumer tetap mengonsumsi *event* melalui *group*-nya sendiri. *Offset group* Notifier tertinggal, sedangkan *event* tetap tersimpan di Kafka sesuai *retention policy* dari *broker*. Setelah Notifier dijalankan kembali menggunakan *group* ID yang sama, *consumer* melanjutkan dari *offset* terakhir yang telah di-*commit* dan memproses *event* yang *published* selama masa berhenti.



Gambar 9. Alur *consumer* berhenti dan menyusul *event*

Consumer ketiga dapat ditambahkan dengan membuat *group* ID baru pada *topic* yang sama. Perubahan ini tidak memerlukan perubahan kode, *rebuild*, ataupun *restart* Aggregator karena *producer* hanya bergantung pada kontrak *topic*. *Consumer* baru dapat memilih membaca *event* baru saja atau membaca kembali *event* yang masih tersedia dengan memulai dari *offset* paling awal.

Alasan Keputusan

Beberapa alternatif distribusi *event* yang dipertimbangkan adalah sebagai berikut.

No.	Alternatif solusi	Kelebihan	Kekurangan
1	Aggregator memanggil setiap *consumer* melalui REST.	Implementasi awal sederhana dan respons *consumer* langsung diketahui.	*Producer* mengetahui seluruh *consumer*, *consumer* lambat menahan alur *producer*, dan penambahan *consumer* mengubah kode Aggregator.

2	RabbitMQ dengan *exchange* dan *durable queue* per *consumer*.	Routing fleksibel, manual *acknowledgement* tersedia, dan sesuai untuk antrean kerja berlatensi rendah.	Setiap *subscriber* memerlukan *queue* tersendiri yang harus di-*provision* dan dipertahankan. *Replay event* lama bagi *subscriber* yang baru dibuat tidak sealami model log Kafka.
3	Kafka dengan *topic* dan consumer *group*.	*Event* disimpan sebagai log, *offset* setiap *group* independen, *consumer* yang berhenti dapat menyusul, dan *group* baru dapat ditambahkan tanpa mengubah *producer*.	Operasional *broker* lebih berat, konsistensi bersifat *eventual*, dan *consumer* harus menangani duplikasi.

Kafka dipilih karena kebutuhan utama bukan sekadar pengiriman pesan sekali jalan, melainkan *fan-out* independen, *event retention*, pengukuran lag, dan kemampuan *consumer* yang berhenti untuk menyusul. Model *consumer group* juga sesuai dengan rencana penambahan *subscriber* baru karena setiap fungsi bisnis memperoleh *offset* sendiri tanpa pengetahuan dari Aggregator.

Konsekuensinya adalah sistem tidak menjanjikan *exactly-once*. *Publisher* dapat mengirim ulang ketika hasil *publish* tidak diketahui, dan *consumer* dapat menerima ulang ketika berhenti sebelum *commit offset*. Pilihan *at-least-once* lebih sesuai untuk kejadian bahaya daripada *at-most-once* karena kehilangan peringatan lebih berisiko daripada menerima *event* yang sama kembali. Risiko duplikasi dipindahkan ke *consumer* dan ditangani dengan transaksi lokal serta *idempotent key*.

Batasan yang Tidak Ditangani

Dalam perancangan Problem 5, terdapat beberapa hal yang secara sadar belum ditangani.

- Kafka masih berjalan sebagai satu *broker* tanpa *replication factor* antar-*node*. Konfigurasi ini cukup untuk memperagakan *fan-out*, *independent offset*, dan *catch-up*, tetapi belum menyediakan *high availability* apabila *broker* gagal.
- Penyimpanan Canonical Store dan publikasi Kafka merupakan dua operasi terpisah. Implementasi menyimpan *event* terlebih dahulu, lalu memublikasikannya. Kegagalan *publish* membuat *cursor polling* tidak maju sehingga *batch* akan dicoba

kembali, tetapi belum ada *transactional outbox* yang memberikan jaminan atomik antara PostgreSQL dan Kafka untuk kondisi kegagalan yang lebih kompleks.

- *Event* yang tidak dapat di-*decode* atau memiliki versi kontrak yang tidak didukung dicatat lalu *offset*-nya di-*commit* agar satu *poison message* tidak menghentikan partisi. *Dead-letter topic* dan proses perbaikan *event* belum tersedia sehingga *event* tidak valid tidak diproses ulang secara otomatis.
- Kontrak *event* memiliki "schema_version", tetapi belum menggunakan *schema registry* atau pemeriksaan kompatibilitas otomatis. *Consumer* hanya menerima versi 1, sehingga evolusi kontrak selain penambahan *field* yang kompatibel memerlukan versi baru dan pembaruan *consumer*.
- Dashboard *Consumer* dan Notifier Consumer masing-masing menggunakan satu *instance* dengan basis data bbolt lokal. Menjalankan beberapa *replica* dalam *group* yang sama akan membagi *event* antar-*instance* sehingga setiap *database* bbolt hanya memiliki sebagian keadaan. Skalabilitas horizontal untuk *materialized view* memerlukan *shared storage* atau strategi partisi yang dirancang khusus.
- *Notifier* hanya membentuk dan menyimpan *record* notifikasi sebagai bukti efek domain. Integrasi dengan *push notification*, SMS, atau perangkat lapangan nyata belum diimplementasikan karena berada di luar pembuktian *fan-out* dan ketahanan *consumer*.
- Lama *retention* dan kapasitas Kafka masih menggunakan konfigurasi *broker* lokal. *Consumer* hanya dapat menyusul selama *event* dan *offset*-nya masih tersedia, sehingga selanjutnya diperlukan *retention policy* dan kapasitas berdasarkan waktu pemulihan maksimum yang disepakati.

Bukti Penyelesaian

Berikut adalah bukti penyelesaian Problem 5.

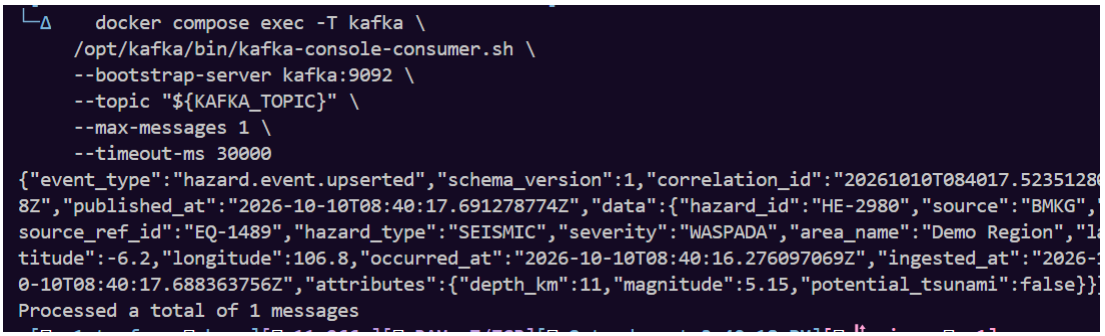
1. **Aggregator *publish* "HazardEvent" ke Kafka**

   Pengujian diawali dengan melakukan pengecekan apakah Aggregator berhasil melakukan *publish* terhadap beberapa *event* ke Kafka.

   Berikut adalah hasil dari *testing*:



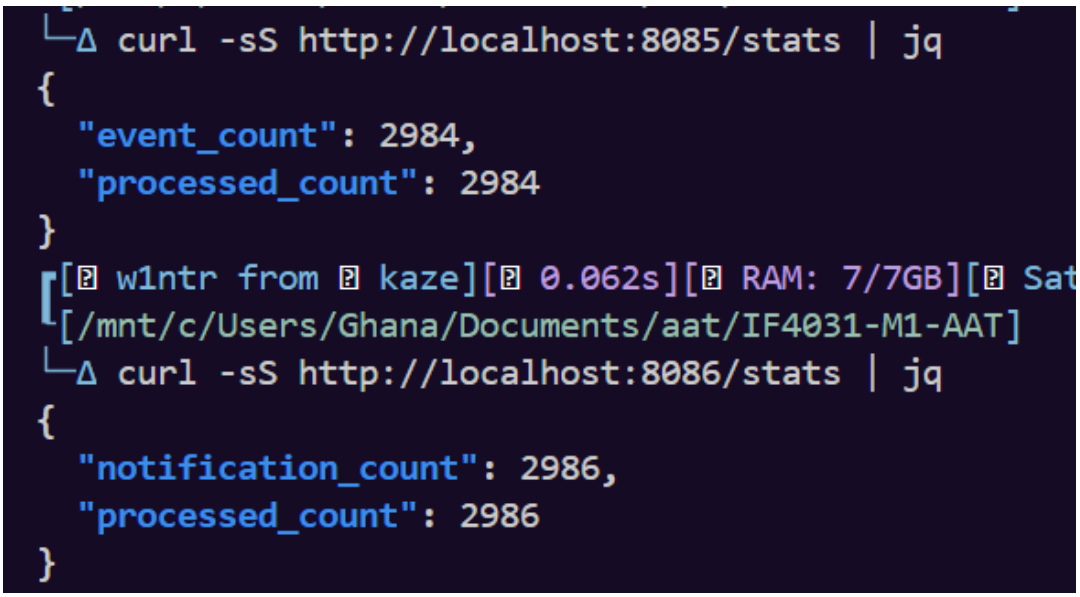
Log di atas membuktikan bahwa Aggregator berhasil melakukan *publish* beberapa *event* ke Kafka. Selanjutnya, cek apakah *event* benar-benar tersimpan di broker (dengan menggunakan *console consumer*), yakni sebagai berikut:



Dari log di atas, terlihat bahwa *event* benar-benar tersimpan pada *broker* (Kafka).

2. *Independent Consumer* menerima *stream* yang sama

Pengujian diawali dengan melakukan pengecekan pada statistik kedua *consumer* untuk *making sure* bahwa masing-masing telah memproses event.



Kedua *consumer* berhasil untuk memproses *event*. Selanjutnya, cari apakah *event* yang sama diterima oleh Aggregator, Dashboard, dan Notifier (gambar di bawah sudah *pre-filtered*, yakni pada “hazard_id”: “HE-2996” dan “correlation_id”: “20261010T084137.525381092Z”):



Sehingga pengujian berhasil.

3. Penghentian Salah Satu *Consumer* Selama Beberapa Waktu

Pengujian diawali dengan melakukan pengecekan pada statistik kedua *consumer* untuk *making sure* bahwa masing-masing telah memproses event.

```
└─Δ curl -sS http://localhost:8085/stats | jq
{
  "event_count": 3170,
  "processed_count": 3170
}
┌[ w1ntr from  kaze][ 0.047s][ RAM: 7/7GB][ Sat
└[/mnt/c/Users/Ghana/Documents/aat/IF4031-M1-AAT]
└─Δ curl -sS http://localhost:8086/stats | jq
{
  "notification_count": 3170,
  "processed_count": 3170
}
```

Selanjutnya, penulis akan menguji dengan mematikan Notifier Consumer, sehingga *make sure* bahwa *lag* awal dari Notifier bernilai 0:

```
└─Δ docker compose exec -T kafka \
  /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server kafka:9092 \
  --describe \
  --group field-notifier-v1

GROUP             TOPIC            PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSU
MER-ID                                                                                        HOST
            CLIENT-ID
field-notifier-v1 hazard.events.v1 0          1083            1083            0               notif
ier-consumer@ac121bda5177 (github.com/segmentio/kafka-go)-a2ecb89e-e3a4-4a7e-a83a-d993fcbdfb30 /172
.20.0.7     notifier-consumer@ac121bda5177 (github.com/segmentio/kafka-go)
field-notifier-v1 hazard.events.v1 1          1063            1063            0               notif
ier-consumer@ac121bda5177 (github.com/segmentio/kafka-go)-a2ecb89e-e3a4-4a7e-a83a-d993fcbdfb30 /172
.20.0.7     notifier-consumer@ac121bda5177 (github.com/segmentio/kafka-go)
field-notifier-v1 hazard.events.v1 2          1040            1040            0               notif
ier-consumer@ac121bda5177 (github.com/segmentio/kafka-go)-a2ecb89e-e3a4-4a7e-a83a-d993fcbdfb30 /172
.20.0.7     notifier-consumer@ac121bda5177 (github.com/segmentio/kafka-go)
```

Selanjutnya, simulasikan ketika pertama kali terjadi *outage*:

```
┌[ w1ntr from  kaze][ 0.014s][ RAM: 7/7GB][ Saturda
└[/mnt/c/Users/Ghana/Documents/aat/IF4031-M1-AAT]
└─Δ OUTAGE_STARTED_AT=$(date -u +%Y-%m-%dT%H:%M:%SZ)
┌[ w1ntr from  kaze][ 0.031s][ RAM: 7/7GB][ Saturda
└[/mnt/c/Users/Ghana/Documents/aat/IF4031-M1-AAT]
└─Δ docker compose stop notifier-consumer
[+] Stopping 1/1
 ✓ Container if4031-m1-aat-notifier-consumer-1  Stopped
```

Dan cek apakah hanya Notifier yang berhenti:

```
└─Δ docker compose ps -a \
  aggregator dashboard-consumer notifier-consumer
NAME                                  IMAGE                                COMMAND                  SE
RVICE              CREATED          STATUS                       PORTS
if4031-m1-aat-aggregator-1            if4031-m1-aat-aggregator             "/app/aggregator"        ag
gregator           4 hours ago      Up 4 hours (healthy)         8083/tcp
if4031-m1-aat-dashboard-consumer-1    if4031-m1-aat-dashboard-consumer     "/app/dashboard-cons…"   da
shboard-consumer   46 minutes ago   Up 46 minutes (healthy)      0.0.0.0:8085->8085/tcp, [::]:8085->
8085/tcp
if4031-m1-aat-notifier-consumer-1     if4031-m1-aat-notifier-consumer      "/app/notifier-consu…"   no
tifier-consumer    46 minutes ago   Exited (0) 39 seconds ago
```

Aggregator dan Dashboard masih Up sedangkan Notifier sudah Exited. Selanjutnya, biarkan beberapa *event* di-*publish*, dan setelah itu cek juga untuk memastikan Aggregator dan Dashboard tetap berjalan:

```
└─Δ sleep 25
[ w1ntr from  kaze][ 25.016s][ RAM: 7/7GB][ Saturday at 4:01:28 PM][ main ≡  ~1]
[/mnt/c/Users/Ghana/Documents/aat/IF4031-M1-AAT]
└─Δ docker compose logs \
  --since "${OUTAGE_STARTED_AT}" \
  aggregator dashboard-consumer \
  | grep -E 'hazard_event_published|dashboard_event_applied'
dashboard-consumer-1  | {"time":"2026-10-10T08:59:27.777500652Z","level":"INFO","msg":"dashboard_ev
ent_applied","service":"dashboard-consumer","correlation_id":"20261010T085927.523564175Z","message_
id":"hazard.events.v1:0:1089","hazard_id":"HE-3210","applied":true}
dashboard-consumer-1  | {"time":"2026-10-10T08:59:29.174471519Z","level":"INFO","msg":"dashboard_ev
ent_applied","service":"dashboard-consumer","correlation_id":"20261010T085927.523735844Z","message_
id":"hazard.events.v1:2:1048","hazard_id":"HE-3211","applied":true}
dashboard-consumer-1  | {"time":"2026-10-10T08:59:38.072006683Z","level":"INFO","msg":"dashboard_ev
ent_applied","service":"dashboard-consumer","correlation_id":"20261010T085937.524494764Z","message_
id":"hazard.events.v1:1:1069","hazard_id":"HE-3212","applied":true}
dashboard-consumer-1  | {"time":"2026-10-10T08:59:40.060122015Z","level":"INFO","msg":"dashboard_ev
ent_applied","service":"dashboard-consumer","correlation_id":"20261010T085937.524411809Z","message_
id":"hazard.events.v1:0:1090","hazard_id":"HE-3213","applied":true}
dashboard-consumer-1  | {"time":"2026-10-10T08:59:47.789117445Z","level":"INFO","msg":"dashboard_ev
ent_applied","service":"dashboard-consumer","correlation_id":"20261010T085947.523851077Z","message_
id":"hazard.events.v1:2:1049","hazard_id":"HE-3214","applied":true}
dashboard-consumer-1  | {"time":"2026-10-10T08:59:50.169225879Z","level":"INFO","msg":"dashboard_ev
ent_applied","service":"dashboard-consumer","correlation_id":"20261010T085947.523716734Z","message_
id":"hazard.events.v1:1:1070","hazard_id":"HE-3215","applied":true}
dashboard-consumer-1  | {"time":"2026-10-10T08:59:57.713984424Z","level":"INFO","msg":"dashboard_ev
```

Dan juga *event* dan *process*-nya masih tetap bertambah dibandingkan dari sebelum *outage*:

```
└─Δ curl -sS http://localhost:8085/stats | jq
{
  "event_count": 3240,
  "processed_count": 3240
}
```

Setelah itu, cek apakah *event* untuk Notifier di-*hold* di Kafka:

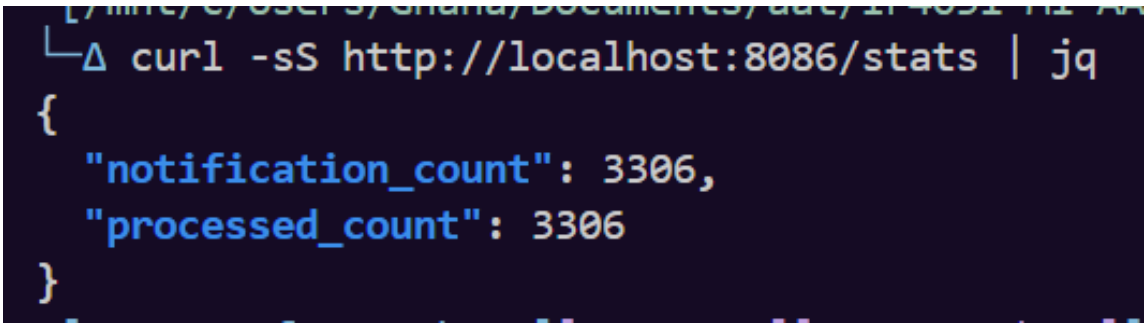
```
└─Δ docker compose exec -T kafka \
  /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server kafka:9092 \
  --describe \
  --group field-notifier-v1

Consumer group 'field-notifier-v1' has no active members.

GROUP             TOPIC            PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSU
MER-ID     HOST            CLIENT-ID
field-notifier-v1 hazard.events.v1 0          1089            1107            18              -
           -               -
field-notifier-v1 hazard.events.v1 1          1069            1082            13              -
           -               -
field-notifier-v1 hazard.events.v1 2          1048            1066            18              -
           -               -
```

Dan juga nilai *lag* sudah bertambah menjadi 18. Setelah ini, nyalakan kembali Notifier, dan tunggu sampai Notifier selesai melakukan *catch up event*:

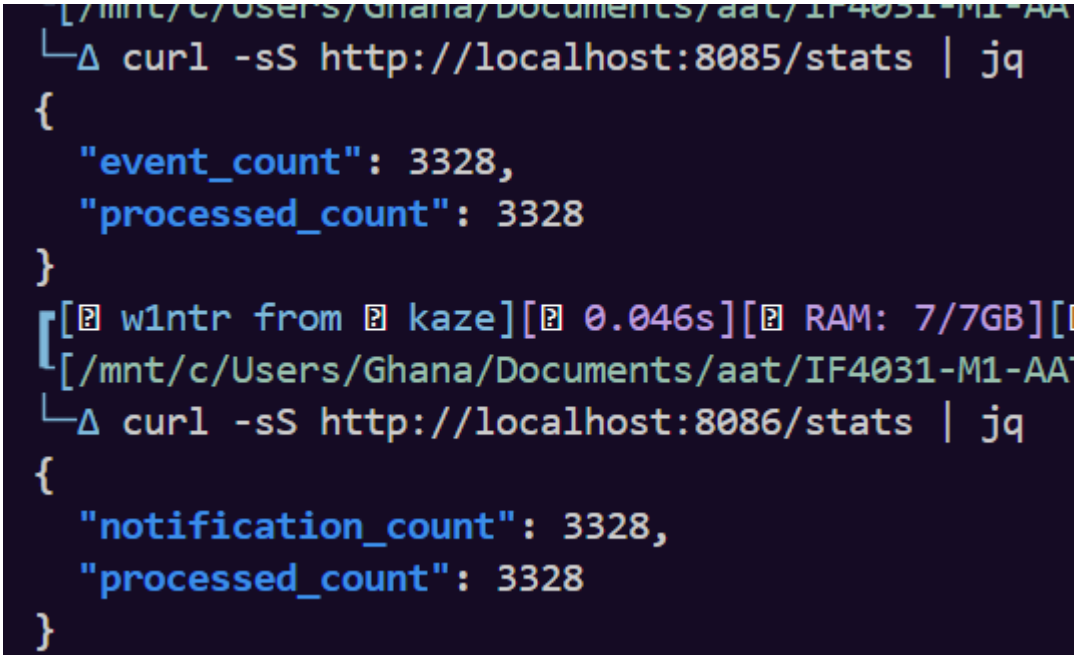
```
┌[ w1ntr from  kaze][ 0.017s][ RAM: 7/7GB][ Saturday at 4:04:41 PM][ main ≡  ~1]
└[/mnt/c/Users/Ghana/Documents/aat/IF4031-M1-AAT]
└─Δ docker compose start notifier-consumer
[+] Running 3/3
 ✓ Container if4031-m1-aat-kafka-1              Healthy                                  1.6s
 ✓ Container if4031-m1-aat-kafka-init-1         Exi...                                   4.5s
 ✓ Container if4031-m1-aat-notifier-consumer-1  Started                                  0.6s
┌[ w1ntr from  kaze][ 7.248s][ RAM: 7/7GB][ Saturday at 4:06:21 PM][ main ≡  ~1]
└[/mnt/c/Users/Ghana/Documents/aat/IF4031-M1-AAT]
└─Δ sleep 20
┌[ w1ntr from  kaze][ 20.016s][ RAM: 7/7GB][ Saturday at 4:06:47 PM][ main ≡  ~1]
└[/mnt/c/Users/Ghana/Documents/aat/IF4031-M1-AAT]
└─Δ curl -sS http://localhost:8086/health | jq
{
  "status": "ok"
}
┌[ w1ntr from  kaze][ 0.052s][ RAM: 7/7GB][ Saturday at 4:06:52 PM][ main ≡  ~1]
└[/mnt/c/Users/Ghana/Documents/aat/IF4031-M1-AAT]
└─Δ docker compose logs \
  --since "${OUTAGE_STARTED_AT}" \
  notifier-consumer \
  | grep -E 'notification_delivered|kafka_commit_completed'
notifier-consumer-1  | {"time":"2026-10-10T09:06:21.928937195Z","level":"INFO","msg":"notification_
delivered","service":"notifier-consumer","correlation_id":"20261010T085927.523735844Z","message_id"
:"hazard.events.v1:2:1048","hazard_id":"HE-3211","delivered":true}
notifier-consumer-1  | {"time":"2026-10-10T09:06:21.935576975Z","level":"INFO","msg":"kafka_commit_
completed","service":"notifier-consumer","correlation_id":"20261010T085927.523735844Z","partition":
2,"offset":1048,"latency_ms":6}
notifier-consumer-1  | {"time":"2026-10-10T09:06:21.938939147Z","level":"INFO","msg":"notification_
delivered","service":"notifier-consumer","correlation_id":"20261010T085947.523851077Z","message_id"
:"hazard.events.v1:2:1049","hazard_id":"HE-3214","delivered":true}
notifier-consumer-1  | {"time":"2026-10-10T09:06:21.946057657Z","level":"INFO","msg":"kafka_commit_
completed","service":"notifier-consumer","correlation_id":"20261010T085947.523851077Z","partition":
2,"offset":1049,"latency_ms":7}
notifier-consumer-1  | {"time":"2026-10-10T09:06:21.949617657Z","level":"INFO","msg":"notification_
delivered","service":"notifier-consumer","correlation_id":"20261010T085957.524242990Z","message_id"
:"hazard.events.v1:2:1050","hazard_id":"HE-3217","delivered":true}
```



Cek juga apakah *lag* kembali menjadi 0:



Dan juga cek kedua statistik *consumer*:



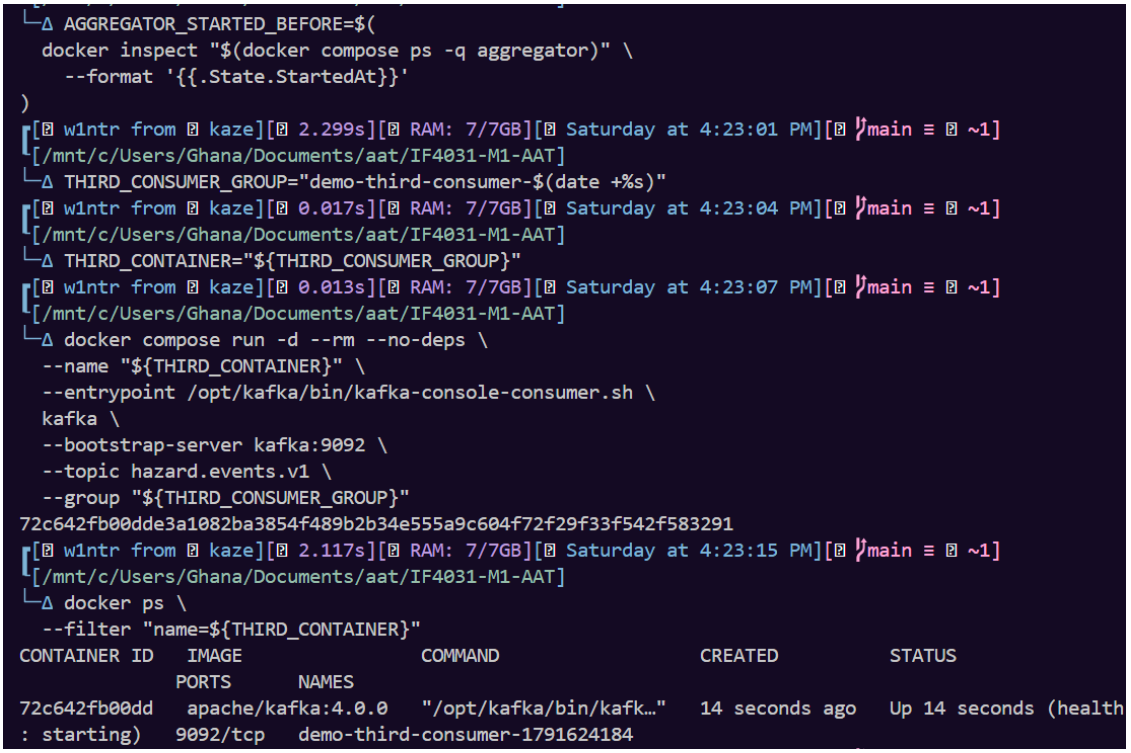
Kesimpulannya, ketika Notifier berhenti, *lag* tersebar sebagai beberapa *event* pada partisi 0, partisi 1, dan partisi 2. Pada waktu yang sama, *log*

“hazard_event_published” dan “dashboard_event_applied” tetap muncul. Setelah Notifier aktif kembali, *log* menunjukkan *event* yang tertinggal diproses lalu diikuti “kafka_commit_completed”. *Lag* seluruh partisi akhirnya kembali menjadi nol.

Dengan demikian, *event* yang terbit selama Notifier berhenti tersusul dan tidak hilang. Kafka mempertahankan *event* pada topic, sedangkan *committed offset consumer group* mencatat posisi terakhir yang telah selesai diproses. Ketika Notifier menggunakan kembali *group* ID “field-notifier-v1”, konsumsi dilanjutkan dari *offset* tersebut hingga *lag* kembali menjadi nol. Jaminan ini berlaku selama *event* masih berada dalam *retention period* dari Kafka.

4. Penambahan *Consumer* Baru Tanpa Mengubah Kode *Producer*

Pengujian diawali dengan mencatat waktu mulai Aggregator relatif terhadap sebelum *consumer* baru dijalankan (dan juga buat nama untuk *consumer group* baru). Setelah itu tambahkan *consumer* barunya ke *service*:



Setelah itu, cek apakah *consumer* baru berhasil menerima *event* dan juga terdaftar di Kafka:

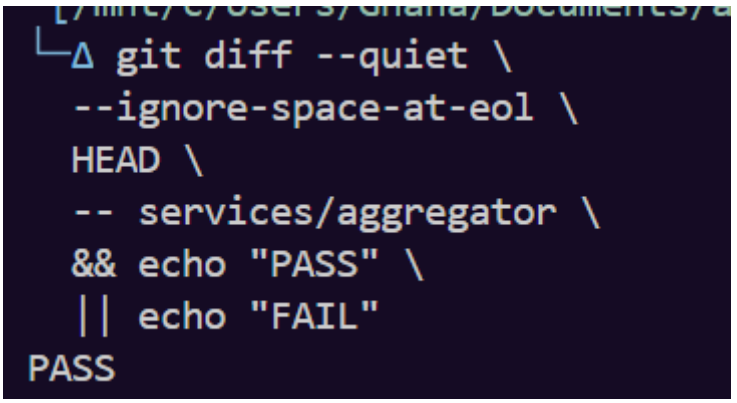
```
└─Δ docker logs -f "${THIRD_CONTAINER}"
{"event_type":"hazard.event.upserted","schema_version":1,"correlation_id":"20261010T092318.313419041Z","published_at":"2026-10-10T09:23:18.574182991Z","data":{"hazard_id":"HE-3498","source":"BMKG","source_ref_id":"EQ-1748","hazard_type":"SEISMIC","severity":"WASPADA","area_name":"Demo Region","latitude":-6.2,"longitude":106.8,"occurred_at":"2026-10-10T09:23:17.065409599Z","ingested_at":"2026-10-10T09:23:18.568581352Z","attributes":{"affected_zones":["Demo Region bagian selatan","Demo Region wilayah pesisir"],"depth_km":14,"estimated_arrival":"2026-10-10T10:08:17.065409599Z","magnitude":5.6,"potential_tsunami":true,"related_event_id":"EQ-1748","threat_level":"Waspada","warning_id":"TW-437"}}}
{"event_type":"hazard.event.upserted","schema_version":1,"correlation_id":"20261010T092318.313505257Z","published_at":"2026-10-10T09:23:19.527389891Z","data":{"hazard_id":"HE-3499","source":"PVMBG","source_ref_id":"VR-1748","hazard_type":"VOLCANIC","severity":"AWAS","area_name":"Gunung Merapi","latitude":-7.54130669258,"longitude":110.446270867,"occurred_at":"2026-10-10T09:23:16.959897004Z","ingested_at":"2026-10-10T09:23:19.525134259Z","attributes":{"ash_column_height_m":900,"eruption_count_24h":2}}}
{"event_type":"hazard.event.upserted","schema_version":1,"correlation_id":"20261010T092328.313855048Z","published_at":"2026-10-10T09:23:28.54852274Z","data":{"hazard_id":"HE-3500","source":"BMKG","source_ref_id":"EQ-1749","hazard_type":"SEISMIC","severity":"WASPADA","area_name":"Demo Region","latitude":-6.2,"longitude":106.8,"occurred_at":"2026-10-10T09:23:27.065409599Z","ingested_at":"2026-10-10T09:23:28.543857975Z","attributes":{"depth_km":15,"magnitude":5.75,"potential_tsunami":false}}}
{"event_type":"hazard.event.upserted","schema_version":1,"correlation_id":"20261010T092328.313909393Z","published_at":"2026-10-10T09:23:28.524499983Z","data":{"hazard_id":"HE-3501","source":"PVMBG","source_ref_id":"VR-1749","hazard_type":"VOLCANIC","severity":"NORMAL","area_name":"Gunung Merapi","latitude":-7.54130669258,"longitude":110.446270867,"occurred_at":"2026-10-10T09:23:26.959897329Z","ingested_at":"2026-10-10T09:23:28.521479788Z","attributes":{"ash_column_height_m":1050,"eruption_count_24h":3}}}
{"event_type":"hazard.event.upserted","schema_version":1,"correlation_id":"20261010T092337.837725457Z","published_at":"2026-10-10T09:23:37.98515952Z","data":{"hazard_id":"HE-3502","source":"BMKG","source_ref_id":"EQ-1750","hazard_type":"SEISMIC","severity":"WASPADA","area_name":"Demo Region","latitude":-6.2,"longitude":106.8,"occurred_at":"2026-10-10T09:23:36.587417814Z","ingested_at":"2026-10
```

```
└─Δ docker compose exec -T kafka \
  /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server kafka:9092 \
  --describe \
  --group "${THIRD_CONSUMER_GROUP}" \
  --members \
  --verbose

GROUP                          CONSUMER-ID                                                 HOST
  CLIENT-ID        #PARTITIONS     CURRENT-EPOCH   CURRENT-ASSIGNMENT     TARGET-EPOCH    TARGET-ASSIGNMENT
demo-third-consumer-1791624184 console-consumer-bba6c746-52ed-4ec0-82c6-f3e2983d5a75 /172.20.0.10
  console-consumer 3              -               hazard.events.v1:0,1,2 -               -
```

Berikut adalah bukti bahwa Aggregator tidak diubah dan juga tidak di-*restart*:

```
└─Δ AGGREGATOR_STARTED_AFTER=$(
  docker inspect "$(docker compose ps -q aggregator)" \
    --format '{{.State.StartedAt}}'
)
[ w1ntr from  kaze][ 2.298s][ RAM: 7/7GB][ Saturday at 4:28:11 PM][
[/mnt/c/Users/Ghana/Documents/aat/IF4031-M1-AAT]
└─Δ [ "${AGGREGATOR_STARTED_BEFORE}" = "${AGGREGATOR_STARTED_AFTER}" ] \
  && echo "PASS" \
  || echo "FAIL"
PASS
```



Terlihat bahwa *consumer* baru berhasil ditambahkan tanpa perlu melakukan *restart* maupun mengubah kode Aggregator.

Jawaban terhadap Butir Permasalahan

Jawaban atas butir kelima Problem 5 adalah sebagai berikut.

- *Broker* yang dipilih adalah Apache Kafka dengan pola *publish-subscribe* berbasis *topic* dan *consumer group*. *Topic* "hazard.events.v1" menjadi kontrak antara Aggregator dan seluruh *subscriber*. Aggregator hanya memublikasikan *event* kanonik ke *topic* tersebut, sedangkan Dashboard, Notifier, dan *consumer* tambahan berlangganan secara independen. Dengan demikian, *producer* tidak perlu mengetahui keberadaan setiap *consumer*.
- Semantik pengiriman untuk *event* valid adalah *at-least-once*. *Consumer* mengambil pesan tanpa *auto-commit*, menyimpan hasil pemrosesan dan *message* ID ke bbolt dalam satu transaksi, kemudian melakukan *commit offset* secara manual. Kegagalan setelah transaksi bbolt selesai tetapi sebelum *offset* berhasil di-*commit* dapat menyebabkan pesan dikirim ulang. Semantik ini dipilih karena dalam domain koordinasi bencana, duplikasi lebih dapat di-*tolerate* dibandingkan dengan kehilangan *event*.
- Konsekuensi semantik *at-least-once* adalah seluruh *consumer* harus bersifat idempoten. Dashboard dan Notifier mencatat identitas pesan berupa "topic:partition:offset" sehingga pesan yang dikirim ulang tidak menerapkan perubahan domain untuk kedua kalinya. Dashboard juga menyimpan keadaan terbaru berdasarkan "hazard_id", sedangkan Notifier menggunakan kombinasi

“event_type:hazard_id:severity” sebagai *key* domain agar *redelivery* tidak menghasilkan notifikasi yang sama.

- *Consumer group* yang berbeda digunakan untuk fungsi bisnis yang berbeda agar masing-masing memperoleh seluruh *stream*. Beberapa *instance* dalam *group* yang sama berbagi partisi sehingga satu pesan hanya diproses oleh satu anggota *group* tersebut. Dengan tiga partisi pada konfigurasi lokal, maksimal tiga anggota dalam satu *group* dapat memproses partisi secara paralel. Kafka hanya menjamin urutan dalam satu partisi, sehingga “hazard_id” digunakan sebagai *message key* agar pembaruan *hazard* yang sama selalu masuk ke partisi yang sama.
- Ketika Notifier berhenti, Kafka tetap menyimpan *event* dan *offset group* Notifier tidak bergerak. Aggregator serta Dashboard tetap berjalan karena tidak menunggu Notifier. Setelah dijalankan kembali menggunakan *group* ID yang sama, Notifier melanjutkan dari *offset* terakhir dan menyusul *event* yang tertinggal selama *event* tersebut masih tersedia dalam *retention period*. *Consumer* baru juga dapat ditambahkan menggunakan *group* ID baru pada *topic* yang sama tanpa mengubah kode, melakukan *rebuild*, atau me-*restart* Aggregator.

Referensi

Parsing Dynamic JSON:

https://oneuptime.com/blog/post/2026-01-23-go-parse-dynamic-json/view

JSON Web Token with Go:

https://medium.com/@folajimiopeyemisax13/authentication-and-authorization-with-go-language-with-jwt-f39c51172833

Kafka with Go:

https://medium.com/@bbeatrice.leung/quick-introduction-to-using-golang-for-kafka-3979e6b6b1aa

Chi with Go:

https://dev.to/arcadebuilds/write-apis-in-go-using-go-chi-d3g

Goroutines:

https://akarinti.tech/id/insight/understanding-go-concurrency-goroutines-and-channels

Service-to-service authentication-authorization patterns:

https://kalpads.medium.com/service-to-service-authentication-authorisation-patterns-e081b078be94

Fan-out mechanism:

https://getstream.io/glossary/fan-out/

https://developer.ibm.com/articles/awb-event-driven-architectures-with-kafka-and-kafka-streams/

Lampiran

Repository pengerjaan tugas besar: https://github.com/Naufal-Pinasthika/IF4031-M1-AAT

Tabel kontribusi:

NIM	**Kontribusi**
13523090	Service-to-Service Auth & Trust (P3), Fan-out Event Distribution (P5), Laporan (sesuai problem)
13524003	Aggregator, Database, Message Broker, Laporan (Deskripsi Umum, Arsitektur Sistem, Problem 1, Problem 4)
13524013	Mock server BMKG & PVMBG, P2 Concurrency, Client facing API, Laporan